# AI for Robotics

## Chapter 2: Kinematics and Simulation

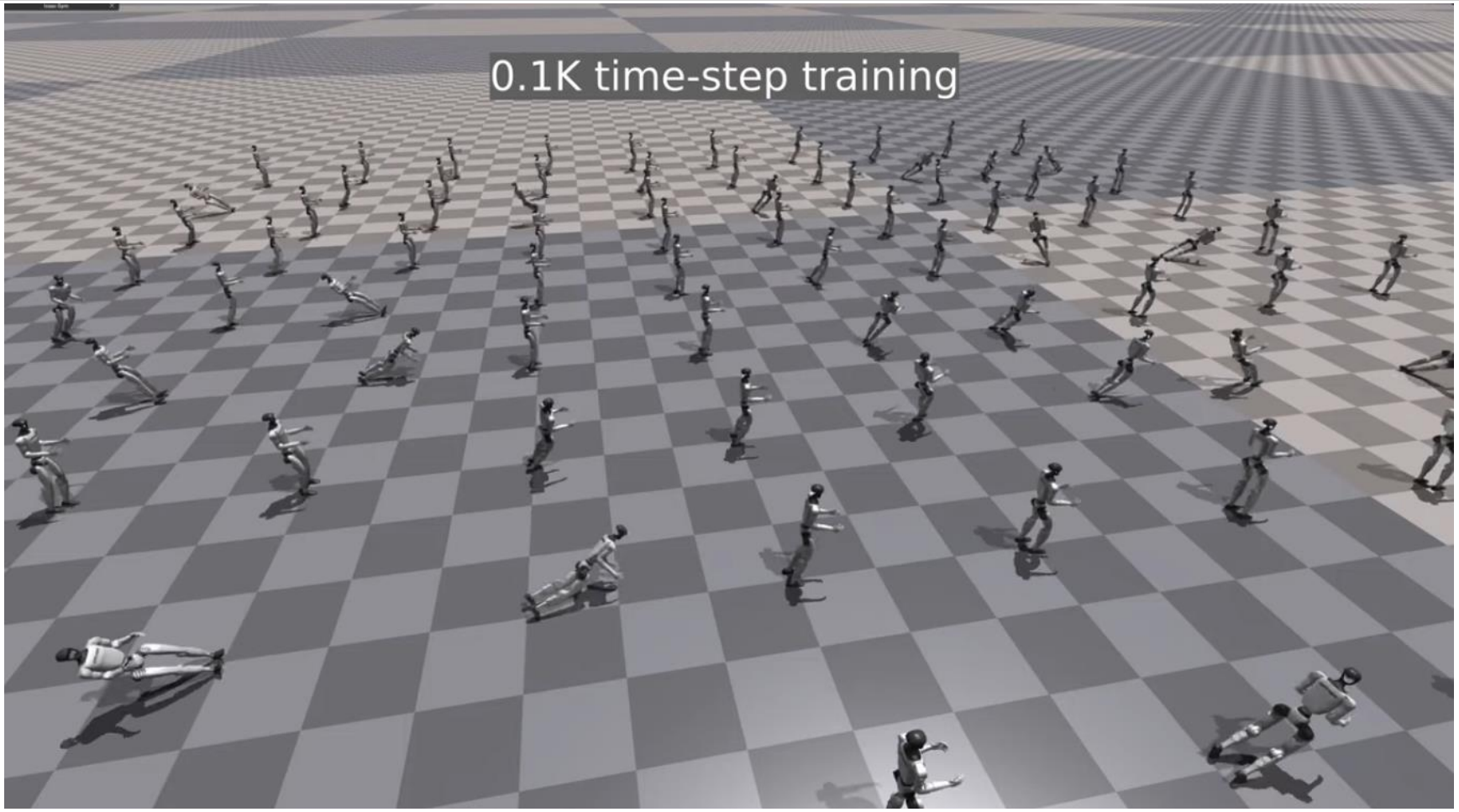
***Instructor: Byungchul Kim***
bc.kim@skku.edu

Many of the contents in this section are adapted from Lynch, K. M., & Park, F. C. (2017). *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press.

# Recap: Robot operation in the real-world is dangerous

https://www.youtube.com/watch?v=67PJ9Neipos

# Recap: Simulation could be an alternatives for real robot operation




- **We don’t need to worry about robot fall down in the simulation**

https://www.youtube.com/watch?v=73hM5ZXuYq0

# Contents (Learning Objectives)

- By the end of today's class, students will be able to understand:

1.1 How to represent the robot motion

- degree of freedom
- types of joints
- coordinate transformation

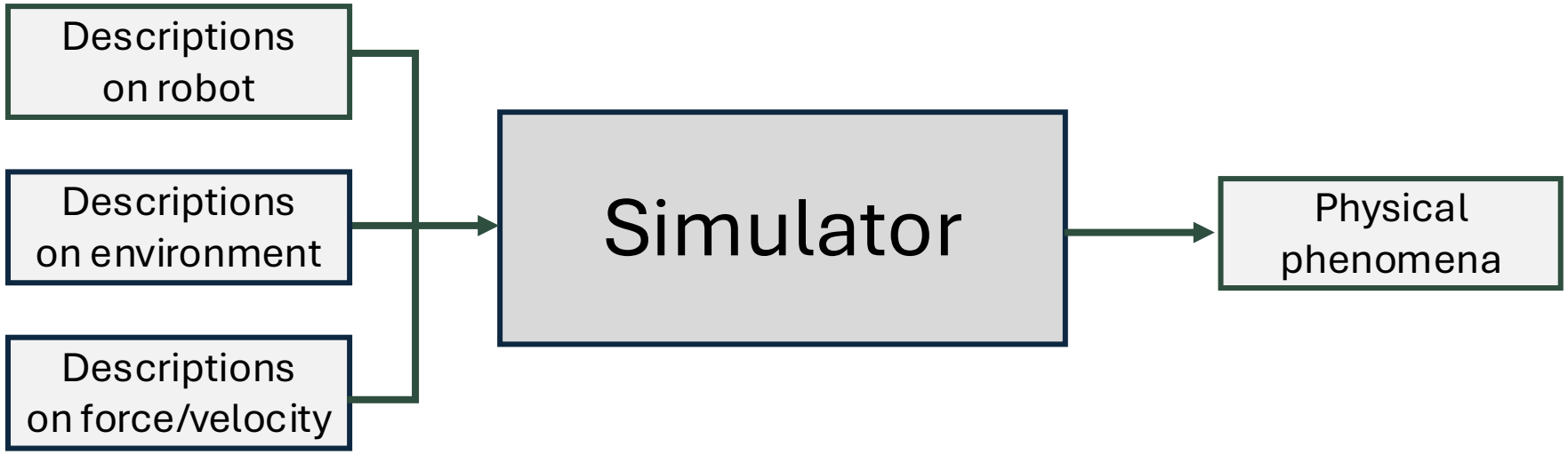
1.2 Format to represent the robot structure in simulator

- famous simulation platforms
- URDF and MJCF

1.3 (if possible) Case study of using simulator in robotic research

# 1. Representation of robot motion

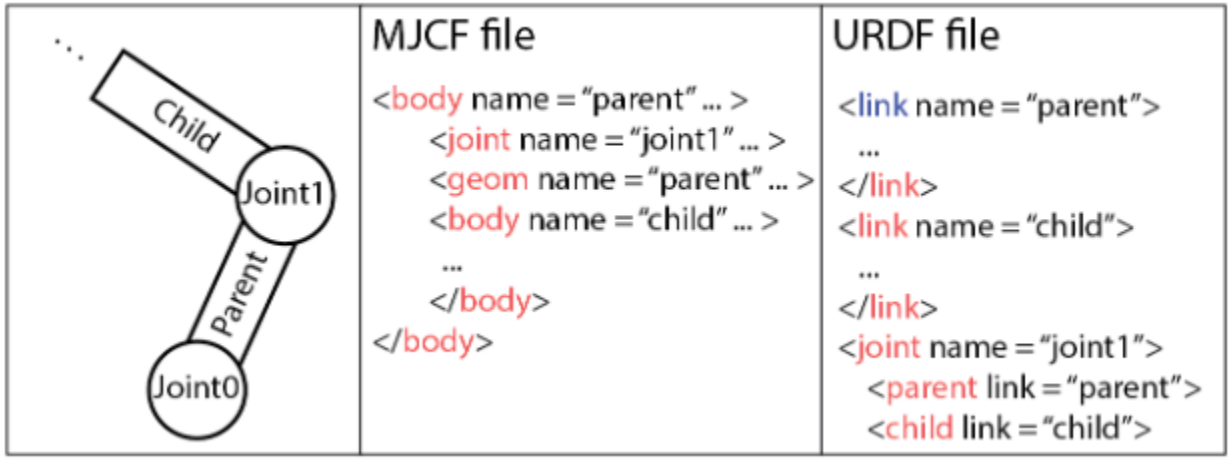
# How Simulator works




- We MUST “represent” the robot in a text format for simulator to understand.
- We MUST “represent” the motion and/or force and/or velocity in several numbers for simulator to understand.

# How can we “represent” the robot in the simulator?

## What kind of “word” or “number” are required to represent the robot state?




MJCF file

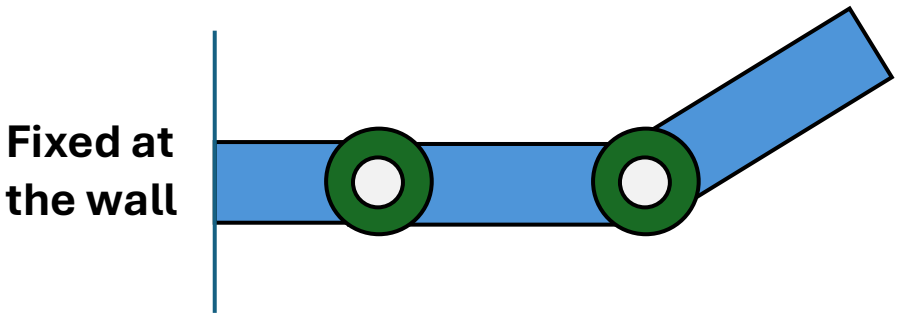
```
<body name = “parent” ... >
    <joint name = “joint1” ... >
    <geom name = “parent” ... >
    <body name = “child” ... >
     ...
    </body>
</body>
```

URDF file

```
<link name = “parent”>
  ...
</link>
<link name = “child”>
  ...
</link>
<joint name = “joint1”>
  <parent link = “parent”>
  <child link = “child”>
```

Robot wrist is heading… front? Robot wrist is heading… down? Robot wrist is heading… left?

# Definitions used by robotic researchers



- **Link:** a rigid component that does not deform (blue part).
- **Joint:** a component that connects two links to make motion (green part).
- **Configuration:** a specification of the positions of all points of the robot.
- **Degree of freedom (DOF):** minimum number required to represent the robot configuration

# Types of joints

## The joints are important in defining the robot motion

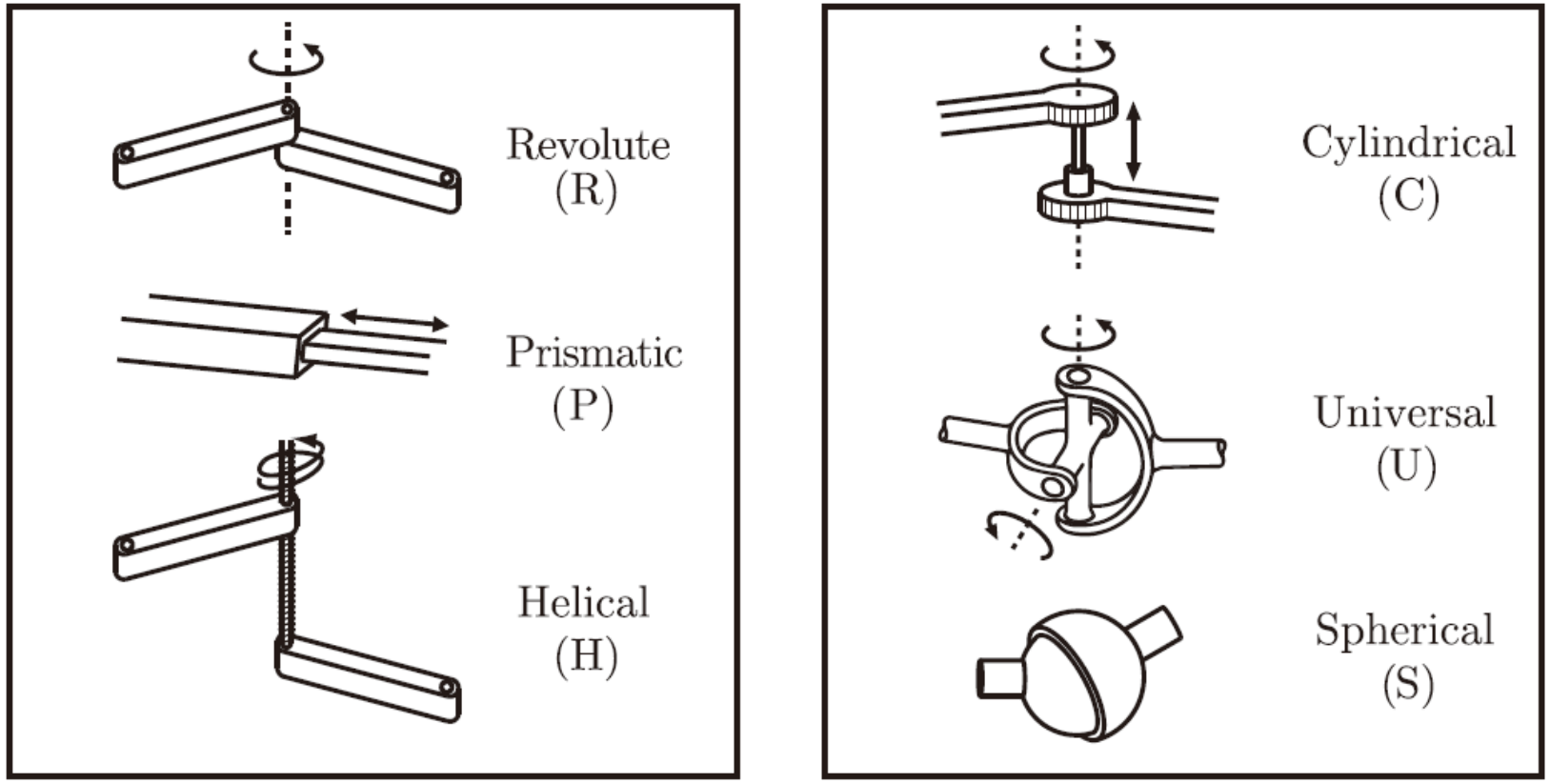
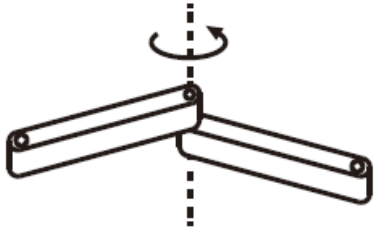


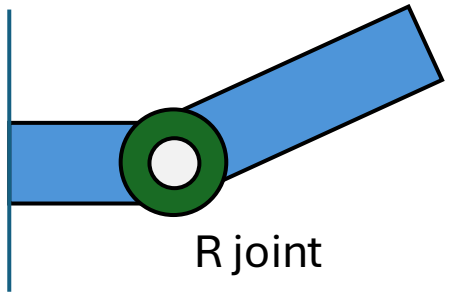

**Figure 2.3:** Typical robot joints.

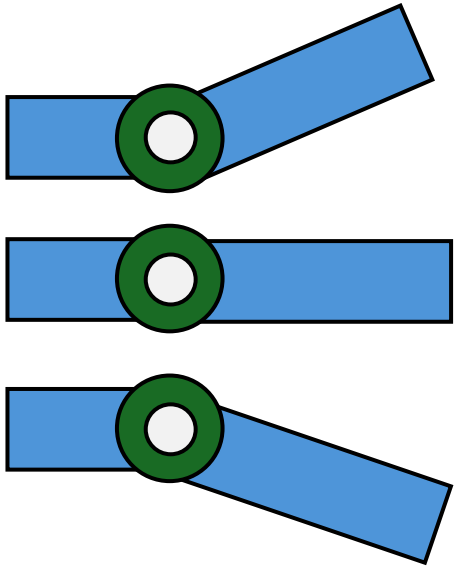
# DOF of the joints

## Each joint types have different degrees of freedom



Revolute
(R)



- **What will be the degree of freedom (DOF) of the R joint?**

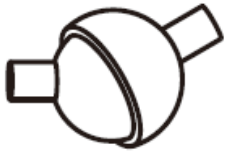
- **Since we need one joint angle to represent the robot configuration, the DOF of the system is 1 (we can also say the joint has 1DOF).**

# DOF of the joints

## Each joint types have different degrees of freedom

Prismatic
(P)

Universal
(U)

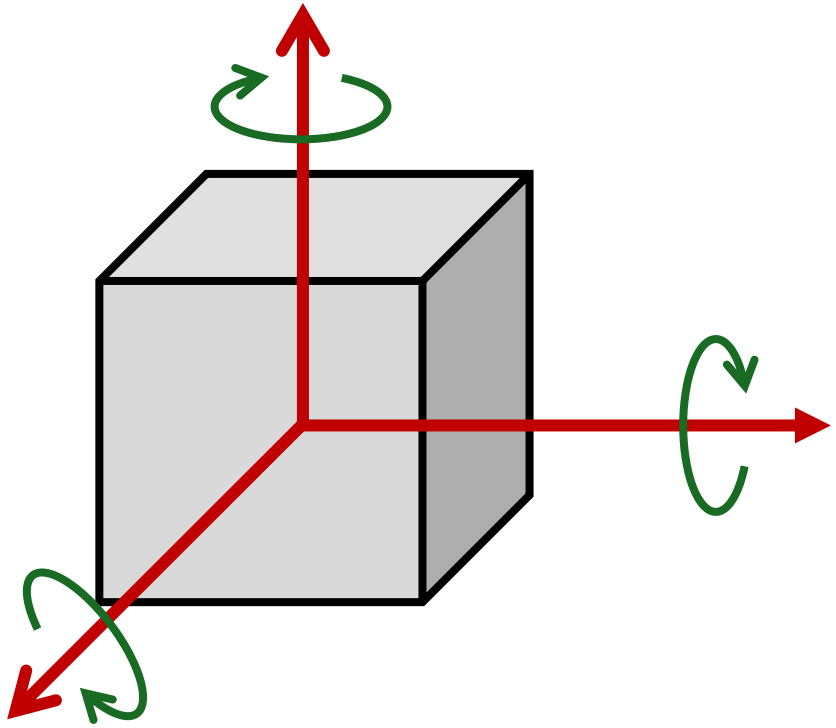


Spherical
(S)

- **What will be the DOF of the P joint?**
- **What will be the DOF of the U joint?**
- **What will be the DOF of the S joint?**

# How about one rigid box in the space?

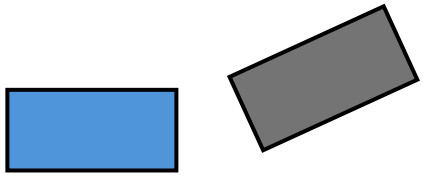
## A rigid box has 6 DOF in the space



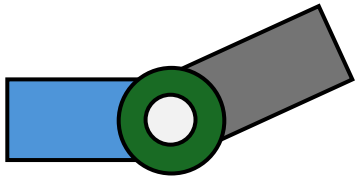
- 3 translational motion (x, y, z)
- 3 rotational motion (yaw, pitch, roll)

# Intuition on joint-link system's DOF

DOF of
the system = 6

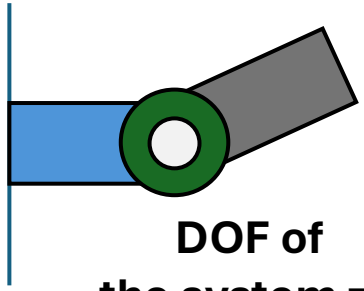


DOF of
the system = 12



DOF of
the system = 7

Fixed at
the wall

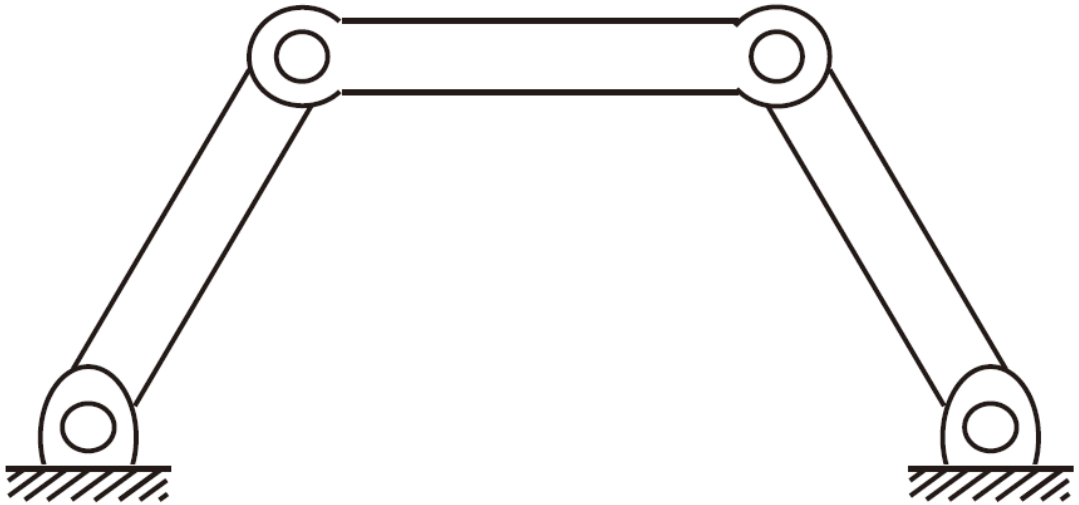


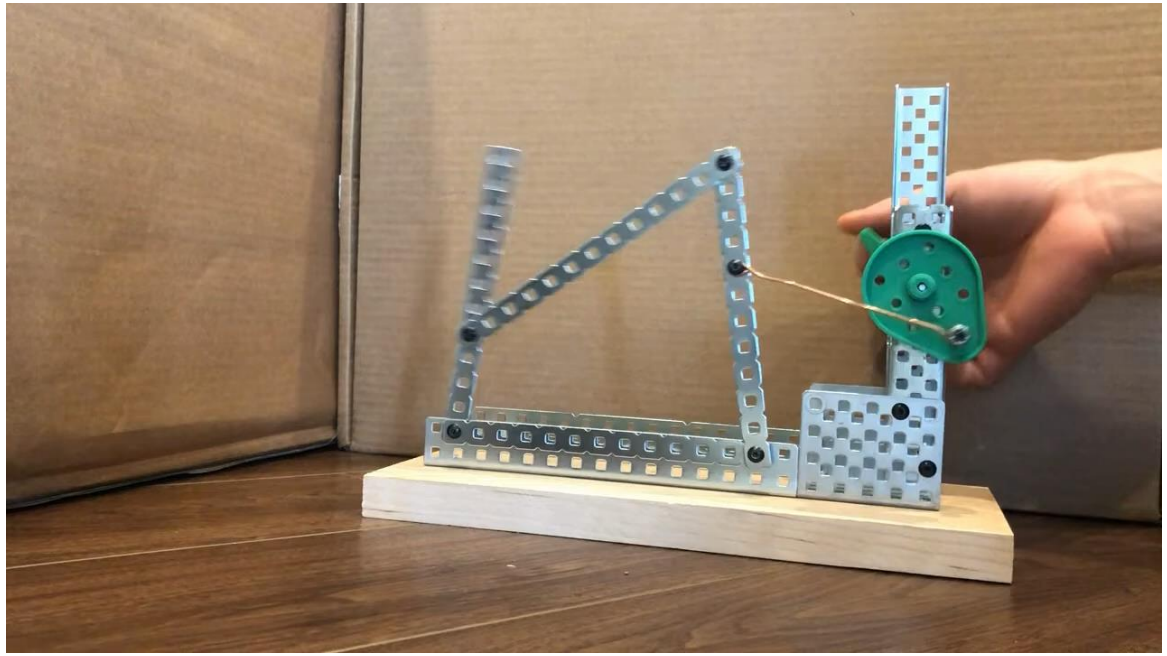
DOF of
the system = 1

- Joint is not "adding" DOF.
- This is rather "constraining" the motion to reduce the DOF.

# Investigating robot system's DOF

## Grubler's Formula





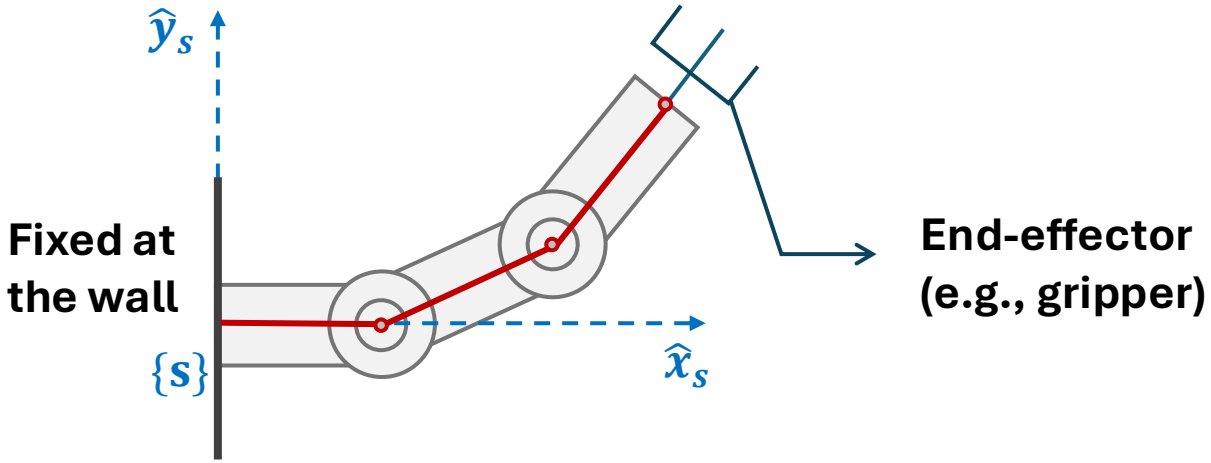
- The DOF of the 4-bar linkage is 1 due to the constraints.
- This can be obtained by Grubler's formula.

https://www.youtube.com/watch?v=1Ty_1LF3Qv0

# 1. Representation of robot motion

# How can we draw a robot in 2D plane?

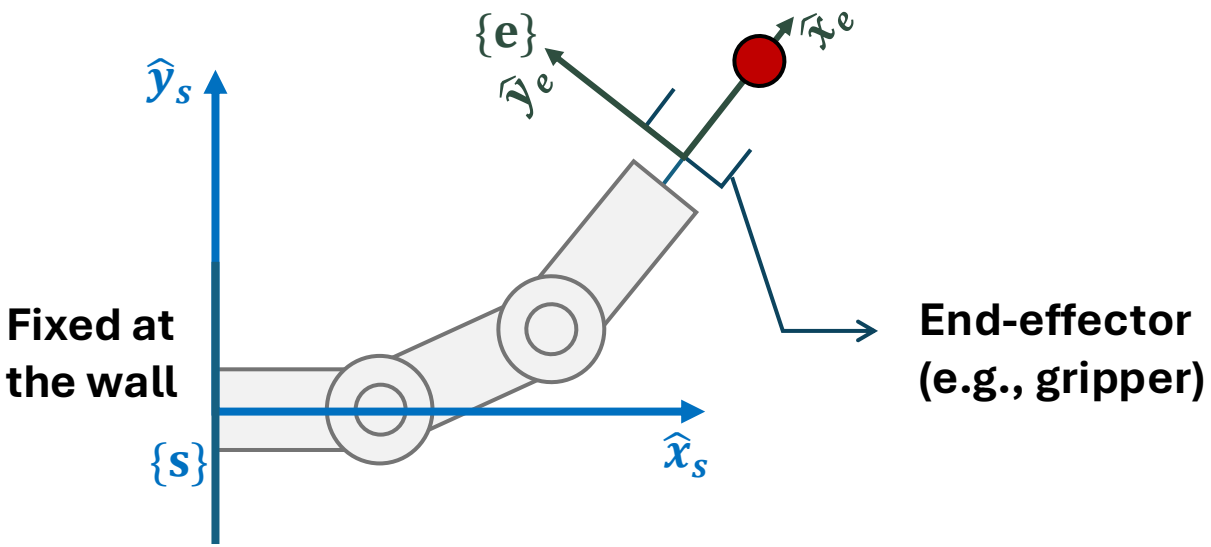
## Assume that we have 2DOF robotic arm with an end-effector




- Goal: Draw robot arm in 2D plane (defined by {s} frame)
  1. Calculate the position of all joints
  2. Connect the line between the joints

# How can we draw an object in 2D plane?

## Assume that we only know object location with respect to end-effector position

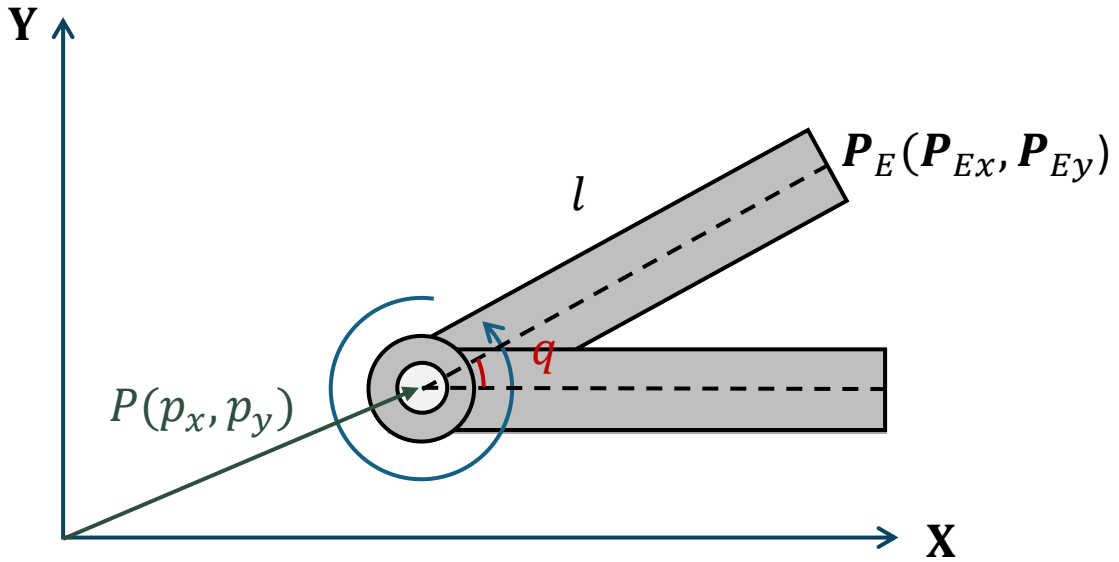


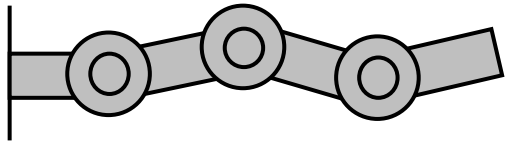

- The object is located at (a,0) in $\{e\}$ frame.
- But how about the position of object in $\{s\}$ frame?

# Forward/Inverse Kinematics

## Mathematical description of motion without considering forces

**Example 1: Robot arm with one revolute joint**



**What if we have more links?**

- Forward kinematics: Finding position and orientation of point $\boldsymbol{P}_E$ from joint angle $q$

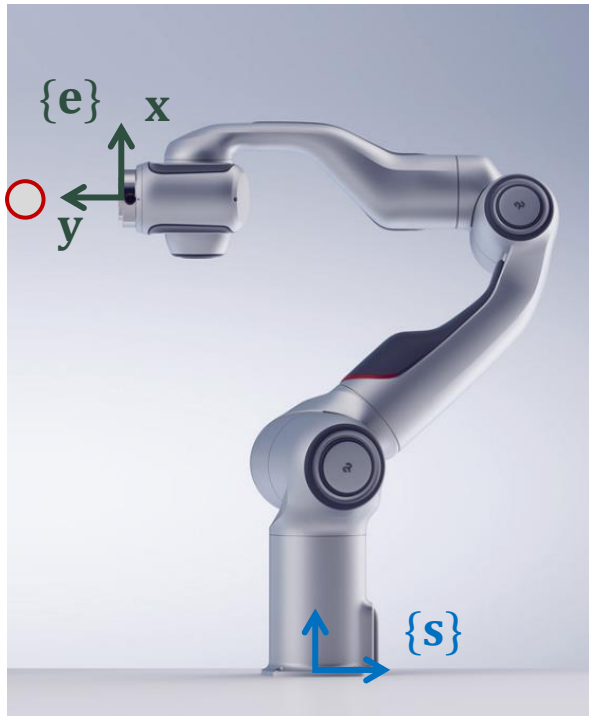
$$\boldsymbol{P}_E = (p_x + l\cos q, p_y + l\sin q)$$

- Inverse kinematics: Finding joint angle $q$ from position and orientation of point $\boldsymbol{P}_E$

$$q = \cos^{-1}[(p_{Ex} - p_x)/l]$$

* Definition itself in google: the branch of mechanics concerned with the motion of objects without reference to the forces which cause the motion.

# Forward kinematics becomes complicated in 3D, multi-link system




- Let object position in $\{e\}$ frame is

$$\boldsymbol{P}_{E,e} = 0\hat{\boldsymbol{x}}_{\boldsymbol{e}} + p_y\hat{\boldsymbol{y}}_{\boldsymbol{e}} + p_z\hat{\boldsymbol{z}}_{\boldsymbol{e}}$$

- How can we represent the object position in $\{s\}$ frame?

$$\boldsymbol{P}_{E,s} = ?\,\hat{\boldsymbol{x}}_{\boldsymbol{s}} + ?\,\hat{\boldsymbol{y}}_{\boldsymbol{s}} + ?\,\hat{\boldsymbol{z}}_{\boldsymbol{s}}$$

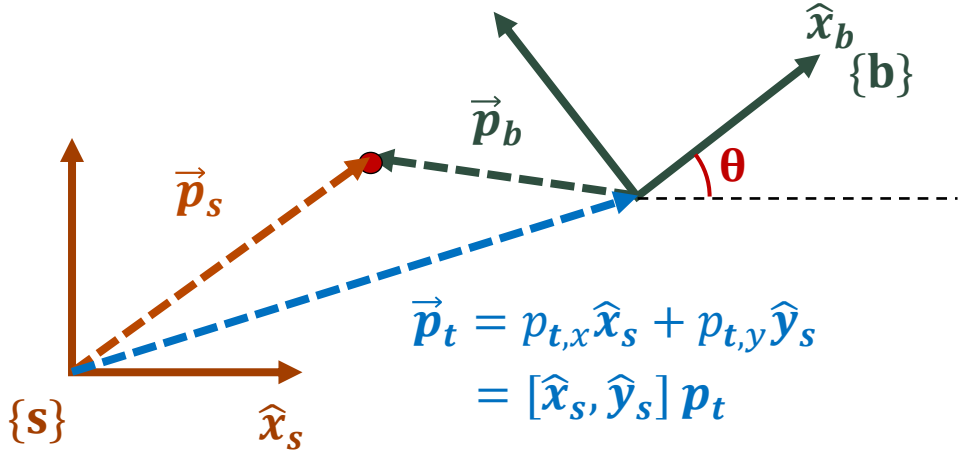
- How can we find joint positions in $\{s\}$ frame?

# Transforming location of point p between two different frames

**Relationship between frame {b} and frame {s}**

$$\hat{x}_b = \cos\theta\ \hat{x}_s + \sin\theta\ \hat{y}_s$$
$$\hat{y}_b = -\sin\theta\ \hat{x}_s + \cos\theta\ \hat{y}_s$$

$$[\hat{x}_b \quad \hat{y}_b] = [\hat{x}_s \quad \hat{y}_s]\begin{bmatrix} C\theta & -S\theta \\ S\theta & C\theta \end{bmatrix}$$
$$= [\hat{x}_s \quad \hat{y}_s]R$$




$$\vec{p}_t = p_{t,x}\hat{x}_s + p_{t,y}\hat{y}_s$$
$$= [\hat{x}_s, \hat{y}_s]\, p_t$$

$$\vec{p}_b = p_{b,x}\hat{x}_b + p_{b,y}\hat{y}_b$$
$$= [\hat{x}_b, \hat{y}_b]\begin{bmatrix} p_{b,x} \\ p_{b,y} \end{bmatrix}$$
$$= [\hat{x}_b, \hat{y}_b]\, p_b$$

$$\vec{p}_s = p_{s,x}\hat{x}_s + p_{s,y}\hat{y}_s$$
$$= [\hat{x}_s, \hat{y}_s]\begin{bmatrix} p_{s,x} \\ p_{s,y} \end{bmatrix}$$
$$= [\hat{x}_s, \hat{y}_s]\, p_s$$

- Using rotation matrix $R$, we can represent the relationship between two frames.

$$\vec{p}_s = \vec{p}_t + \vec{p}_b$$
$$= [\hat{x}_s \quad \hat{y}_s][p_{t,x} \quad p_{t,y}]^T + [\hat{x}_b \quad \hat{y}_b][p_{b,x} \quad p_{b,y}]^T$$
$$= [\hat{x}_s \quad \hat{y}_s][p_{t,x} \quad p_{t,y}]^T + [\hat{x}_s \quad \hat{y}_s]R[p_{b,x} \quad p_{b,y}]^T$$
$$= [\hat{x}_s \quad \hat{y}_s](p_t + Rp_b)$$

$$p_s = p_t + Rp_b$$

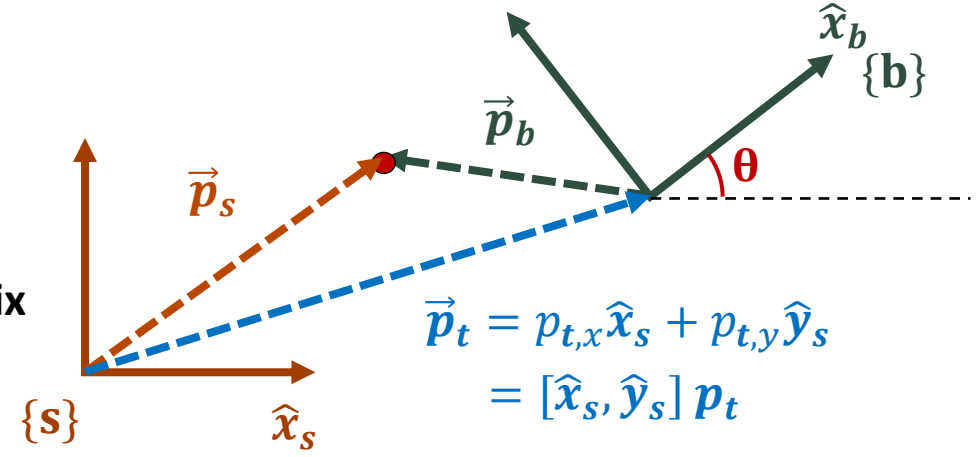
In {b} frame,
$\vec{p}_b$ is represented as $[p_{b,x}, p_{b,y}]^T$

In {s} frame,
$\vec{p}_b$ is represented as $\vec{p}_s$
which is $[p_{t,x}, p_{t,y}]^T + R[p_{b,x}, p_{b,y}]^T$

# Transforming location of point p between two different frames

$$\vec{p}_s = \vec{p}_t + \vec{p}_b$$
$$= [\hat{x}_s \quad \hat{y}_s](p_t + Rp_b)$$

$$p_s = \underbrace{p_t}_{\text{Translation vector}} + \underbrace{R}_{\text{Rotation matrix}} p_b$$




$$\vec{p}_b = p_{b,x}\hat{x}_b + p_{b,y}\hat{y}_b$$
$$= [\hat{x}_b, \hat{y}_b]\begin{bmatrix} p_{b,x} \\ p_{b,y} \end{bmatrix}$$
$$= [\hat{x}_b, \hat{y}_b]\, p_b$$

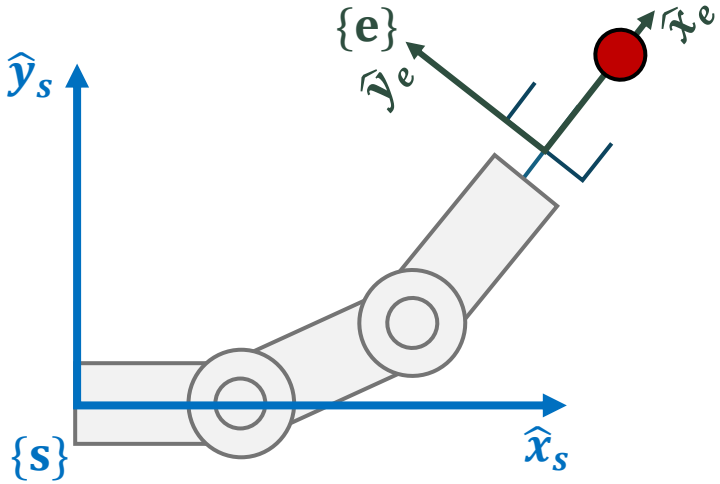
$$\vec{p}_s = p_{s,x}\hat{x}_s + p_{s,y}\hat{y}_s$$
$$= [\hat{x}_s, \hat{y}_s]\begin{bmatrix} p_{s,x} \\ p_{s,y} \end{bmatrix}$$
$$= [\hat{x}_s, \hat{y}_s]\, p_s$$

- Let $[p_{b,x}, p_{b,y}, 1]^T$ and $[p_{s,x}, p_{s,y}, 1]^T$ as $\tilde{p}_b$ and $\tilde{p}_s$ (usually called homogeneous form)
- Then, we can rewrite $p_s = p_t + Rp_b$ as

$$\tilde{p}_s = \underbrace{\boxed{\begin{bmatrix} R & p_t \\ 0 & 1 \end{bmatrix}}}_{\text{Transformation matrix } (T_{sb})} \tilde{p}_b$$

# Recap: How can we draw an object in 2D plane?

## Assume that we only know object location with respect to end-effector position



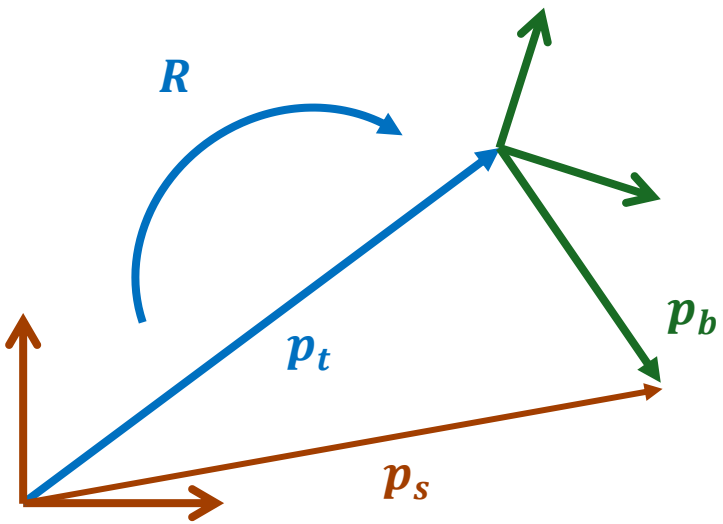

The object is located at (a,0) in $\{e\}$ frame.

- What is the position of object in $\{s\}$ frame?

$$\tilde{p}_s = \begin{bmatrix} R & p_t \\ 0 & 1 \end{bmatrix} \tilde{p}_e = T_{se}\tilde{p}_e$$

$$\tilde{p}_e = [a, 0, 1]^T$$

# Key takeaway in coordinate transformation




$$\tilde{p}_{\boldsymbol{s}} = T_{bs}\tilde{p}_{\boldsymbol{b}}$$

$$\tilde{p}_{\boldsymbol{s}} = [p_{s,x}, p_{s,y}, 1]^T$$

$$\tilde{p}_{\boldsymbol{b}} = [p_{b,x}, p_{b,y}, 1]^T$$

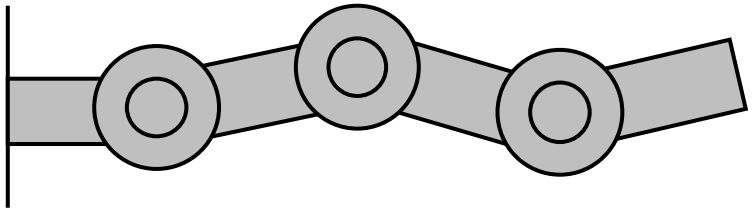
$$T_{sb} = \begin{bmatrix} R & p_t \\ 0 & 1 \end{bmatrix}$$

- **Once we know rotation matrix $R$ and translation vector $p_t$, we can transform the coordinate!**

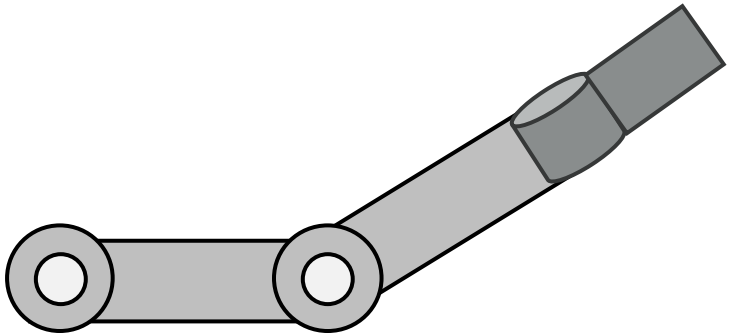
# Expansion of the previous example

## Can we do more complex things?

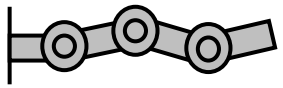
**1. Number of joints can be increased**



**2. Arm can move in 3D space**



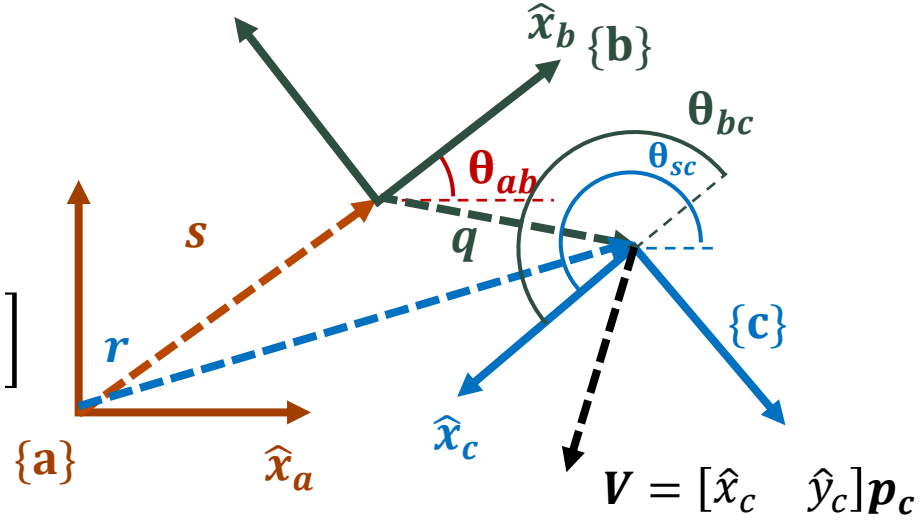
# Transforming location of point p between three different frames



$$\tilde{p}_{\boldsymbol{a}} = T_{ab}\tilde{p}_{\boldsymbol{b}}$$

$$T_{ab} = \begin{bmatrix} R_{ab} & s \\ 0 & 1 \end{bmatrix}$$

$$R_{ab} = \begin{bmatrix} \cos\theta_{ab} & \sin\theta_{ab} \\ -\sin\theta_{ab} & \cos\theta_{ab} \end{bmatrix}$$




$$\tilde{p}_{\boldsymbol{b}} = T_{bc}\tilde{p}_{\boldsymbol{c}}$$

$$T_{bc} = \begin{bmatrix} R_{bc} & q \\ 0 & 1 \end{bmatrix}$$

$$R_{bc} = \begin{bmatrix} \cos\theta_{bc} & \sin\theta_{bc} \\ -\sin\theta_{bc} & \cos\theta_{bc} \end{bmatrix}$$

$$\tilde{p}_{\boldsymbol{a}} = T_{ac}\tilde{p}_{\boldsymbol{c}}$$

$$T_{ac} = \begin{bmatrix} R_{ac} & r \\ 0 & 1 \end{bmatrix}$$

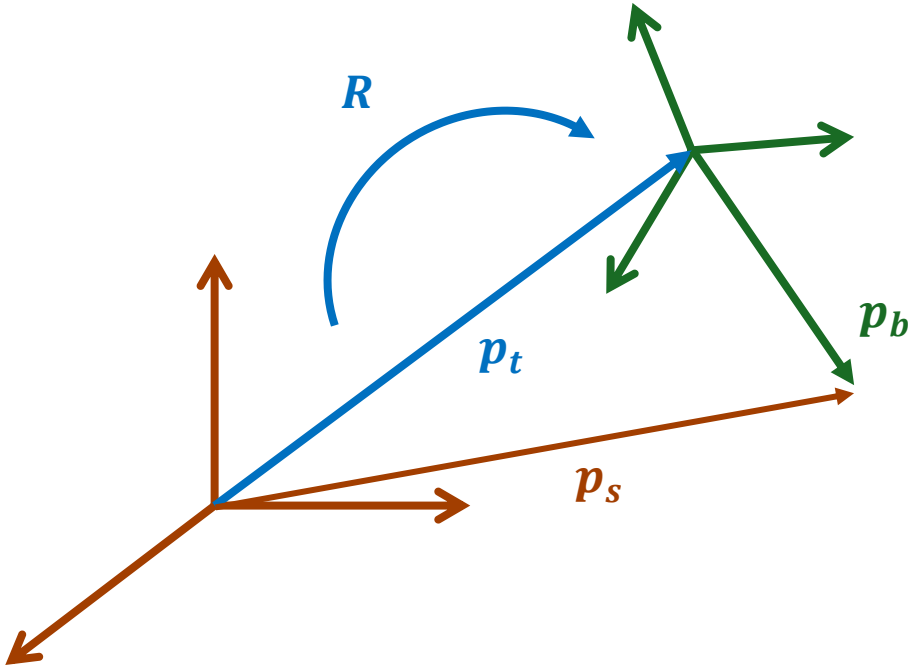
$$R_{ac} = \begin{bmatrix} \cos\theta_{ac} & \sin\theta_{ac} \\ -\sin\theta_{ac} & \cos\theta_{ac} \end{bmatrix}$$

$$\tilde{p}_{\boldsymbol{a}} = T_{ab}\tilde{p}_{\boldsymbol{b}} = T_{ab}T_{bc}\tilde{p}_{\boldsymbol{c}}$$

$$T_{ac} = T_{ab}T_{bc}$$

$$T_{0N} = T_{01}T_{12}\cdots T_{N-1N}$$

# How about in 3D?




$$\tilde{p}_{\boldsymbol{s}} = T_{bs}\tilde{p}_{\boldsymbol{b}}$$

$$\tilde{p}_{\boldsymbol{s}} = [p_{s,x}, p_{s,y}, p_{s,z}, 1]^T$$

$$\tilde{p}_{\boldsymbol{b}} = [p_{b,x}, p_{b,y}, p_{b,z} 1]^T$$

$$T_{sb} = \begin{bmatrix} R & p_t \\ 0 & 1 \end{bmatrix}$$

$$R = \begin{bmatrix} r_{11} & r_{21} & r_{31} \\ r_{12} & r_{22} & r_{32} \\ r_{13} & r_{23} & r_{33} \end{bmatrix}$$

- **How do we obtain $R$ in 3D case?**

# Intuition for understanding rotation matrix

## Rotation matrix is just a representation of axis vector

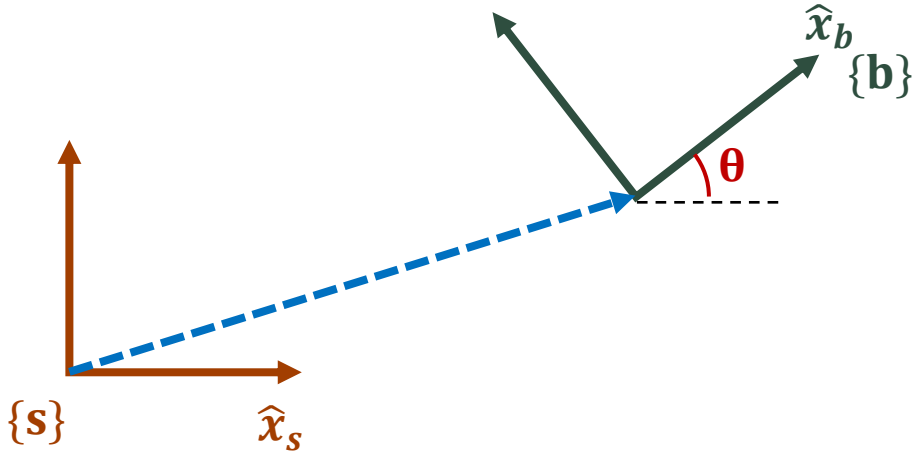
**Relationship between frame $\{\mathbf{b}\}$ and frame $\{\mathbf{s}\}$**

$$\hat{x}_b = \cos\theta\ \hat{x}_s + \sin\theta\ \hat{y}_s$$

$$\hat{y}_b = -\sin\theta\ \hat{x}_s + \cos\theta\ \hat{y}_s$$

$$[\hat{x}_b \quad \hat{y}_b] = [\hat{x}_s \quad \hat{y}_s]\begin{bmatrix} C\theta & -S\theta \\ S\theta & C\theta \end{bmatrix}$$

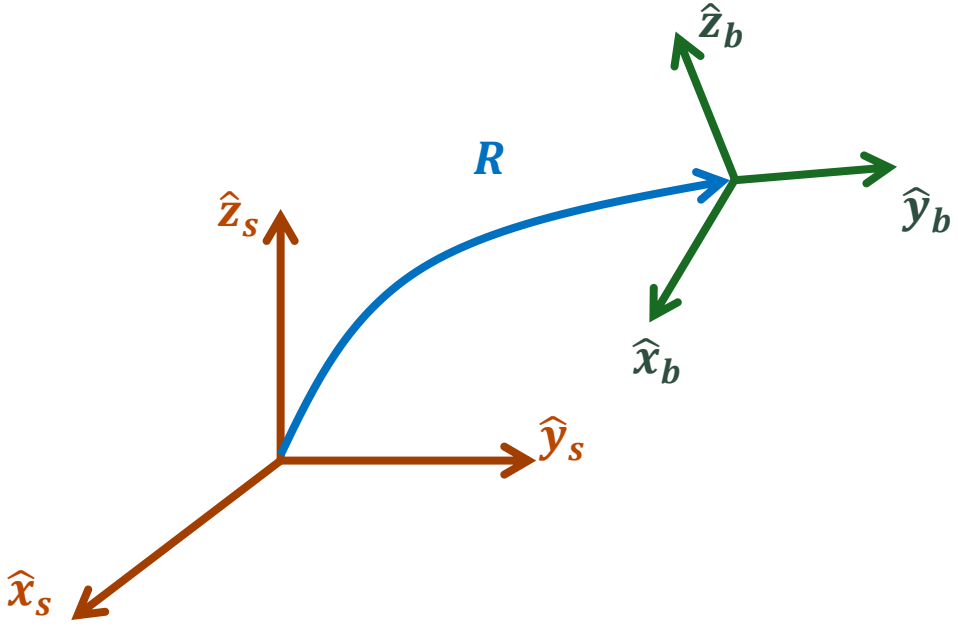
$$= [\hat{x}_s \quad \hat{y}_s]\boldsymbol{R}$$




- **If we find how to represent $\hat{x}_b$ and $\hat{y}_b$ in $\{\mathbf{s}\}$ frame, we can find the rotation matrix.**

# Intuition for understanding rotation matrix

## We can obtain rotation matrix if we know the axis representation




**2D rotation matrix definition**

$$\hat{x}_b = \cos\theta\ \hat{x}_s + \sin\theta\ \hat{y}_s$$
$$\hat{y}_b = -\sin\theta\ \hat{x}_s + \cos\theta\ \hat{y}_s$$

$$[\hat{x}_b \quad \hat{y}_b] = [\hat{x}_s \quad \hat{y}_s]\begin{bmatrix} C\theta & -S\theta \\ S\theta & C\theta \end{bmatrix}$$
$$= [\hat{x}_s \quad \hat{y}_s]R$$

**Assume that**

$$\hat{x}_b = v_{xx}\hat{x}_s + v_{xy}\hat{y}_s + v_{xz}\hat{z}_s$$
$$\hat{y}_b = v_{yx}\hat{x}_s + v_{yy}\hat{y}_s + v_{yz}\hat{z}_s$$
$$\hat{z}_b = v_{zx}\hat{x}_s + v_{zy}\hat{y}_s + v_{zz}\hat{z}_s$$

**Then rotation matrix in 3D can be represented as**

$$R = \begin{bmatrix} v_{xx} & v_{yx} & v_{31} \\ v_{xy} & v_{yy} & v_{zy} \\ v_{xz} & v_{yz} & v_{zz} \end{bmatrix}$$

**It requires nine numbers to represent the rotation matrix.**

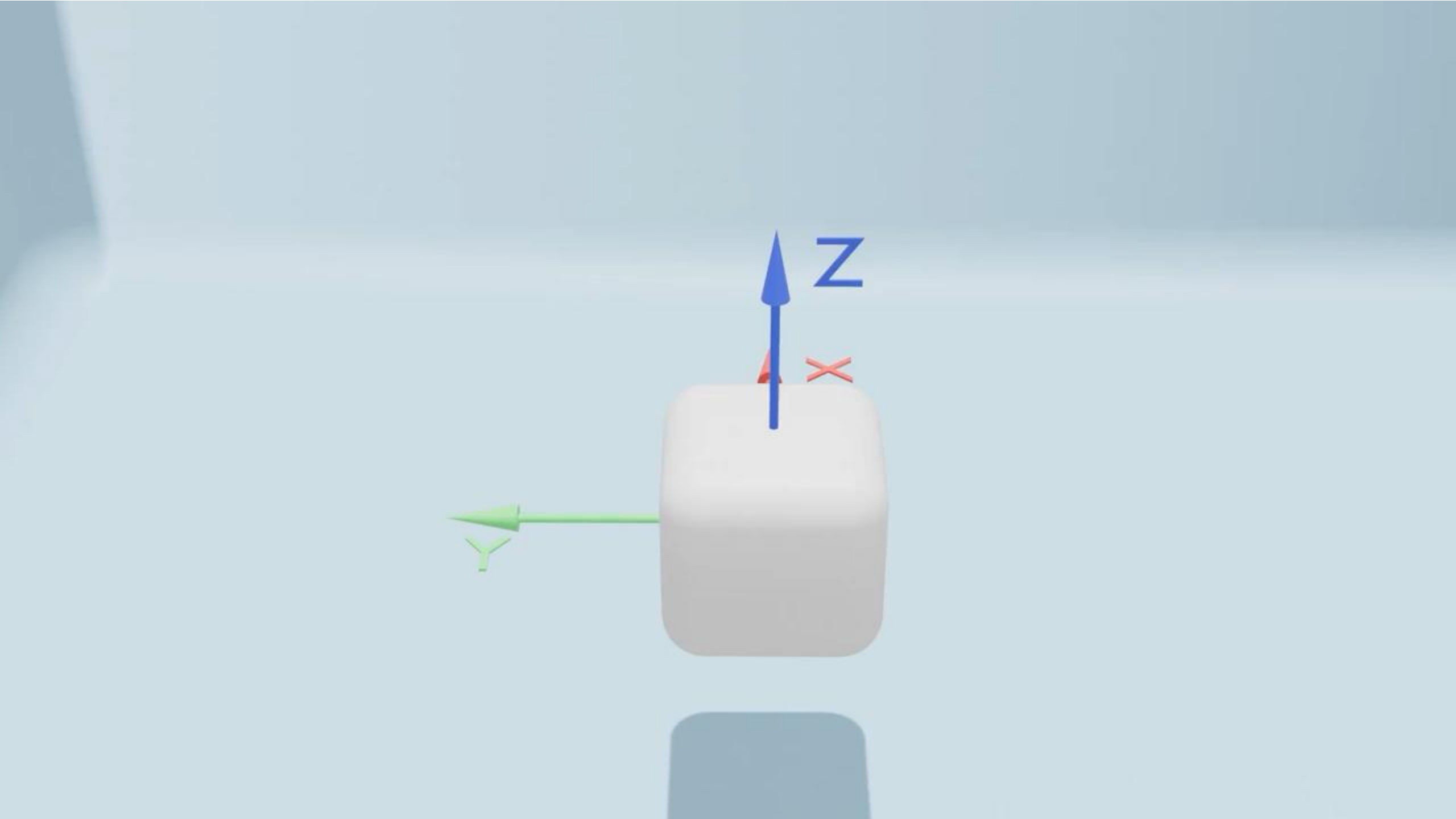
# Can we find better ways to represent rotation matrix?

## Use roll, pitch, and yaw directional rotation to create arbitrary rotation matrix

- **We can rotate in roll, pitch, and yaw directions to create arbitrary rotation matrix**
  - Roll: rotation about the x-axis
  - Pitch: rotation about the y-axis
  - Yaw: rotation about the z-axis

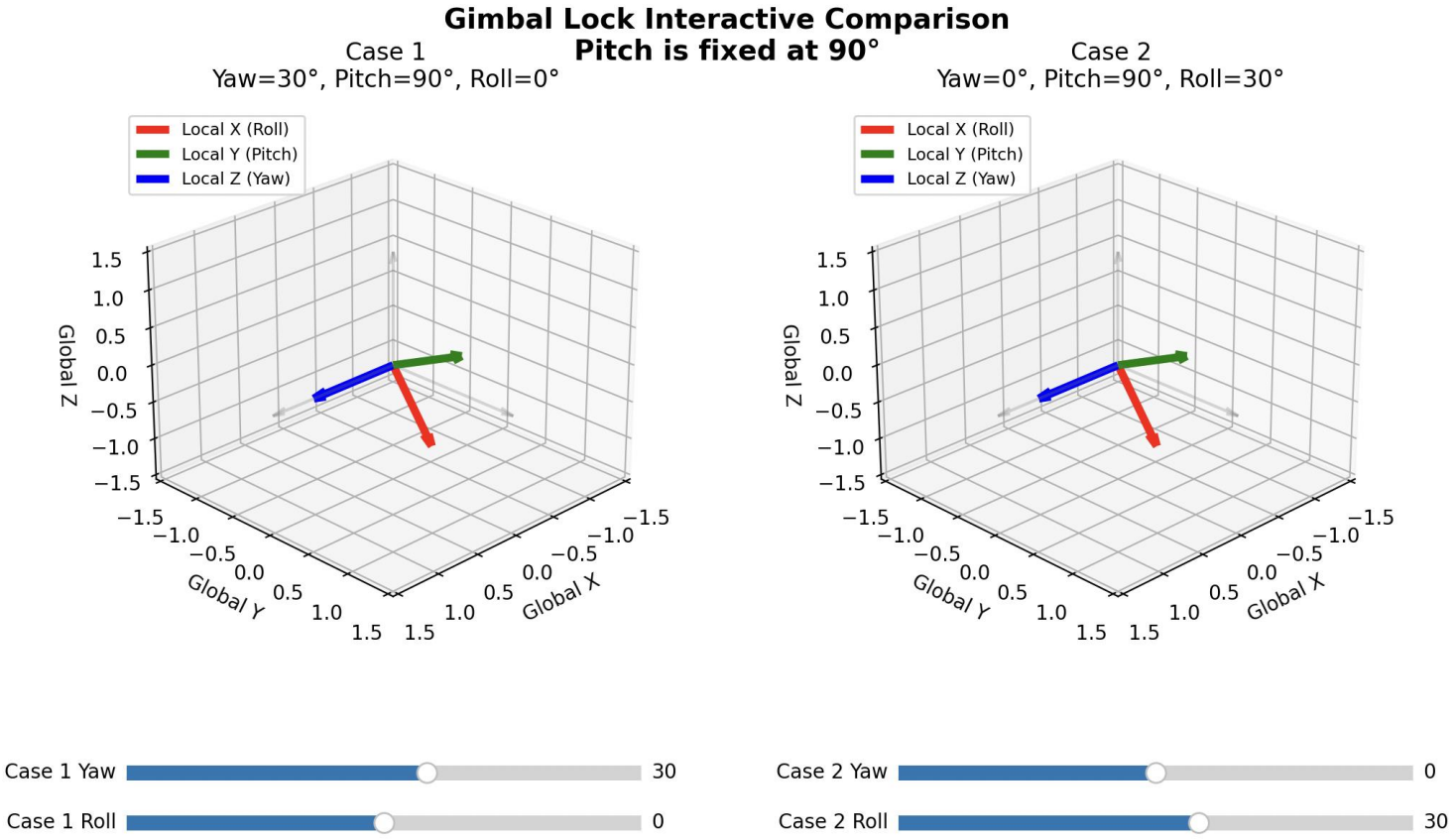
- **We first rotate frame in roll direction (x-axis). Then, we rotate in pitch direction (y-axis). Fin ally, we rotate in yaw direction.**

- **We have “extrinsic” rotation and “intrinsic” rotation.**
  - Extrinsic rotation make rotation with respect to fixed axis, while the intrinsic rotation make rotation with respect to moving axis.


Z
X
Y

# Can we find better ways to represent rotation matrix?

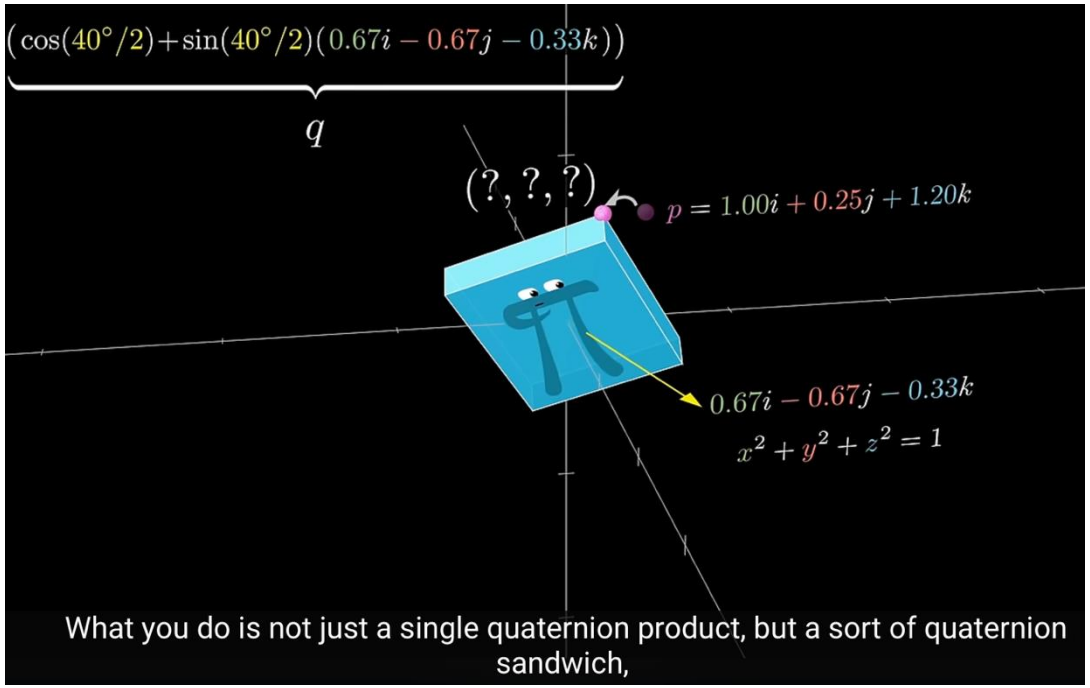
## RPY method has an issue called gimbal lock




- **Two different rpy angle set give the same results.**

# Can we find better ways to represent rotation matrix?

## Quaternion (with four numbers) can solve the gimbal lock issue




- Assume we have one rotational axis $(v_x, v_y, v_z)$ and amount of rotation $\theta$.
- We define one quaternion by $\cos\left(\frac{\theta}{2}\right) + \sin\left(\frac{\theta}{2}\right)(v_x\hat{\imath} + v_y\hat{\jmath} + v_z\hat{k})$
- Use three imaginary numbers and one real number to represent the rotation.

https://www.youtube.com/watch?v=zjMuIxRvygQ

# Summary

- **Robotics require multiple coordinate because body parts move continuously.**

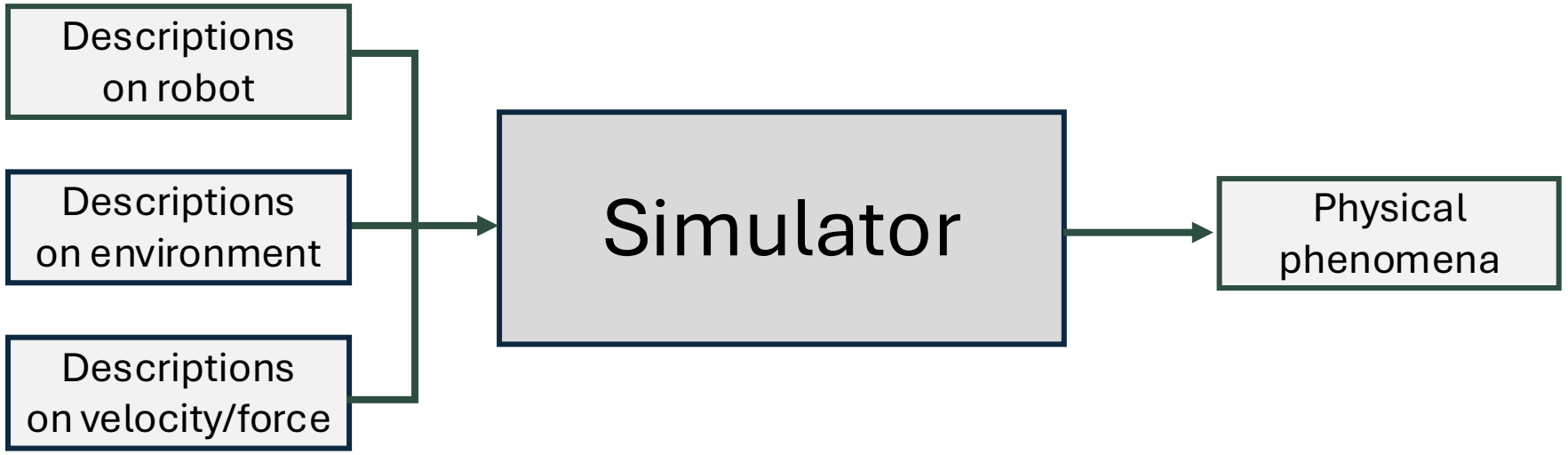
- **We need coordinate transformation to**
  - Know how the robot configuration will be (with given joint angle)
  - Know how the situation is going on at the end-effector

- **Coordinate transformation requires**
  - Rotation matrix and translation vector

- **The most common way to represent rotation matrix includes (1) Euler angle method and (2) Quaternion method.**

# 2. Format to represent the robot in simulation

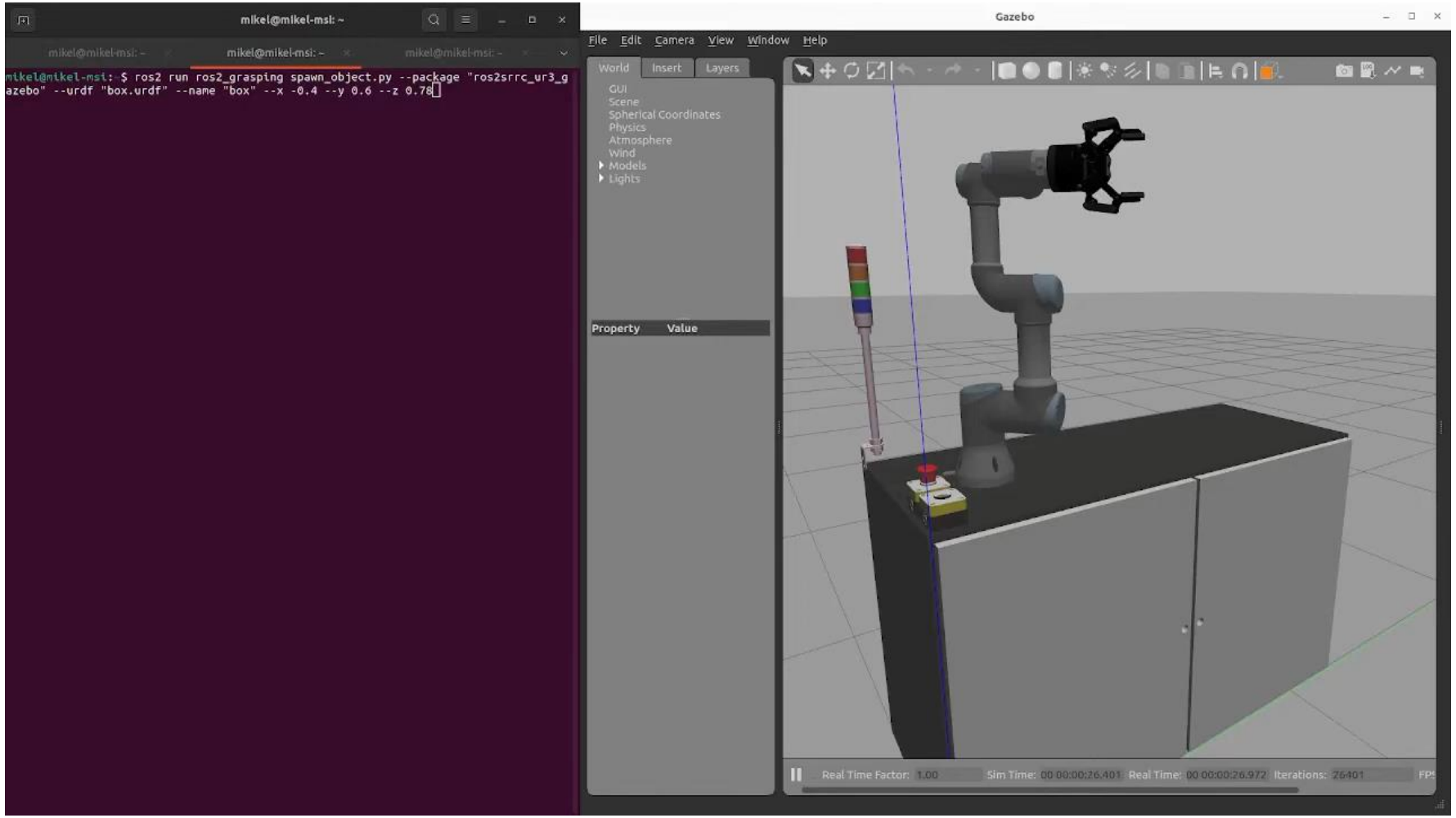
# Recall: How Simulator works




- We MUST “represent” the robot in a text format for simulator to understand.
- We MUST “represent” the motion and/or force and/or velocity in several numbers for simulator to understand.

# Famous simulation platforms

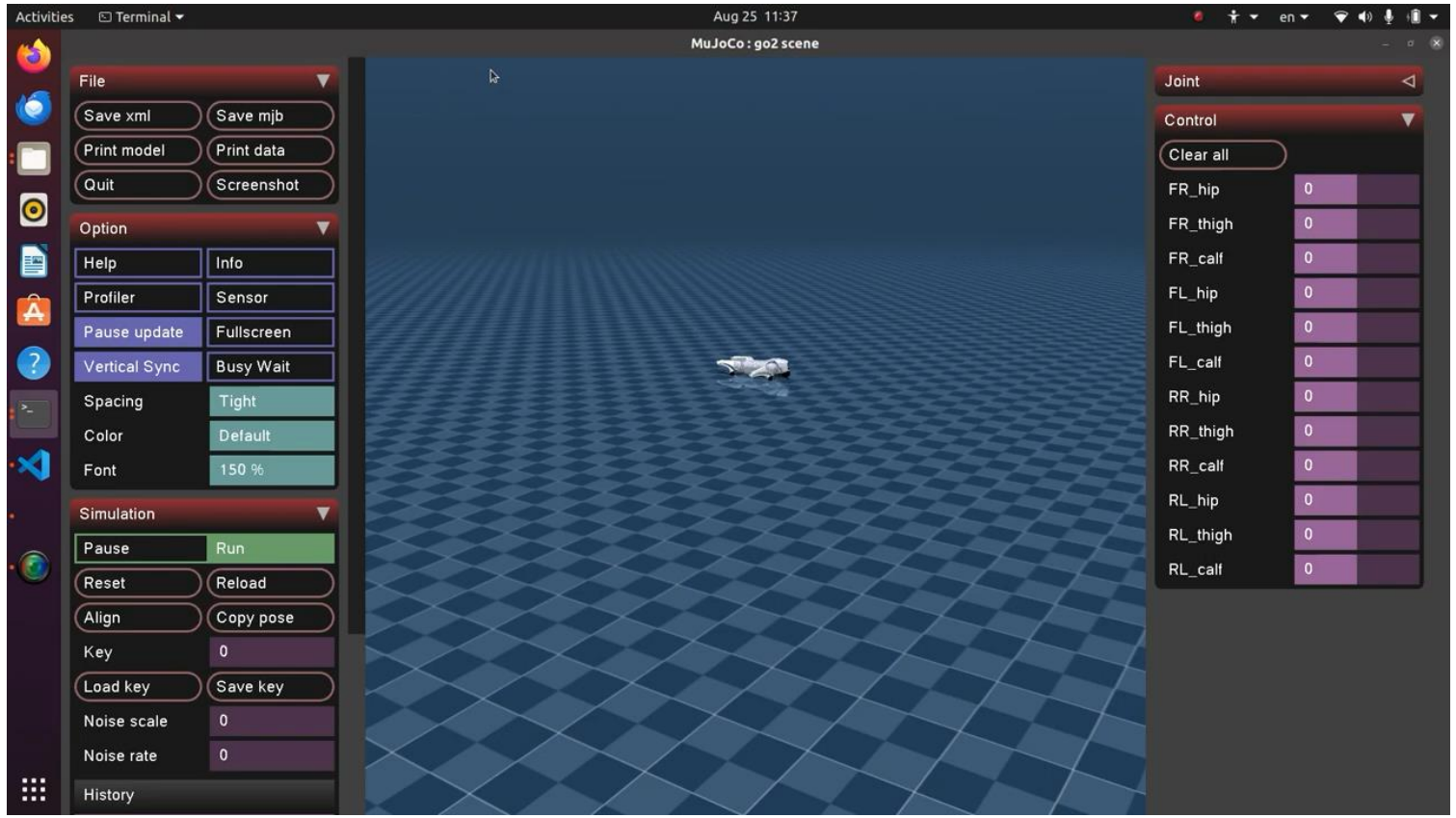
## Gazebo (used in ROS)




- Easy to use in ROS (robot operation system) environment.
- Requires to install ROS.

# Famous simulation platforms

## MuJoCo (Multi-Joint dynamics with Contact)




- **Opensource, (relatively) light**

https://www.youtube.com/watch?v=9VJujzU8dDU

# Famous simulation platforms

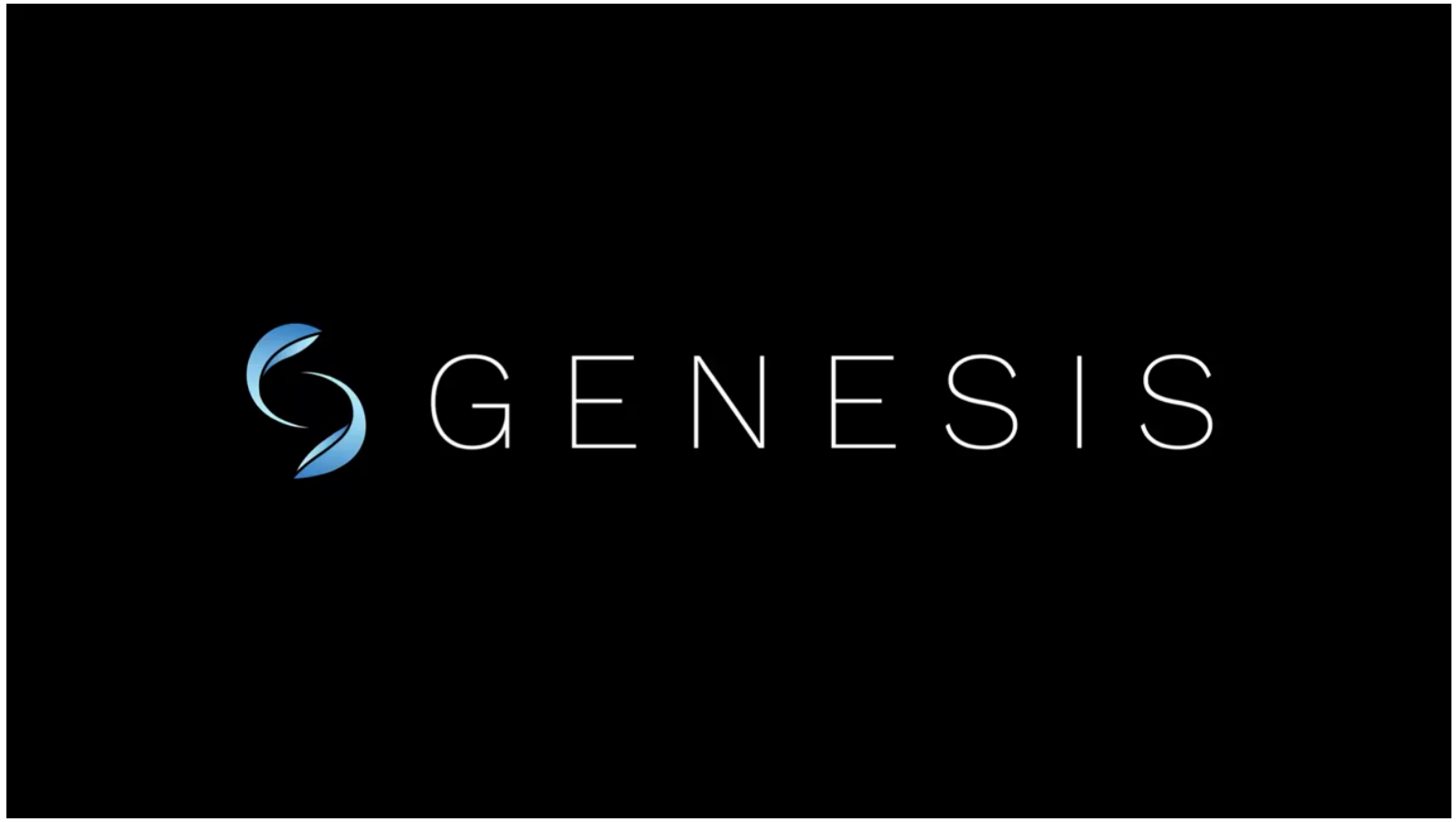
## Issac Sim/Lab

- Good for parallel simulation
- GPU acceleration

https://www.youtube.com/watch?v=VW-dOMBFj7o

# Famous simulation platforms

## Genesis




- Can be generated using language input

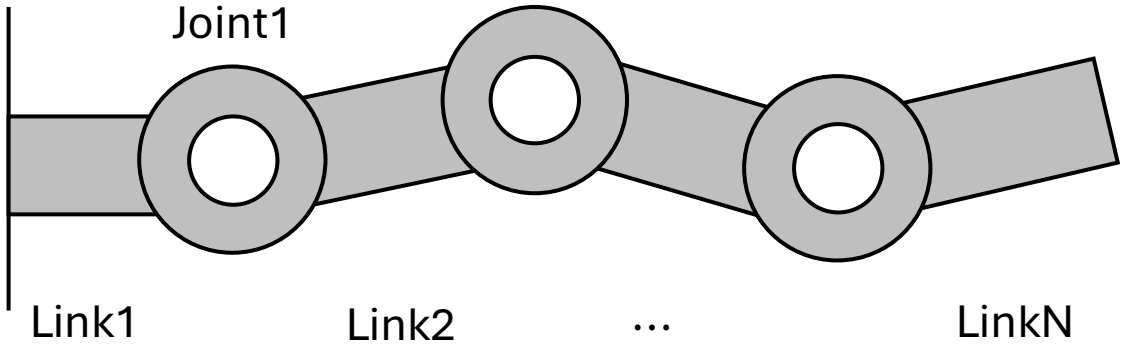
# Pros/Cons of each simulator

## Generated by GPT…

| Simulator | Strengths | Weaknesses | Best Use |
|---|---|---|---|
| **Isaac Sim** | Very realistic rendering, strong GPU acceleration, good for perception | Heavy setup, requires strong GPU, slower physics for control loops | Vision, digital twins, warehouse robots |
| **Genesis** | Fast GPU physics, built for large-scale learning | Ecosystem still young | RL, co-design, large-scale robot training |
| **MuJoCo** | Extremely accurate contact physics, very fast CPU simulation | No native ROS ecosystem, visualization limited | Control research, manipulation |
| **Gazebo** | Native ROS integration, large robotics ecosystem | Slower physics, graphics outdated | ROS robotics systems |

# Universal Robot Description Format (URDF)

## URDF describes link and joint information




$$\tilde{p}_s = \boxed{\begin{bmatrix} \boldsymbol{R} & \boldsymbol{p_t} \\ 0 & 1 \end{bmatrix}} \tilde{p}_b$$

**Transformation matrix ($T_{sb}$)**

```
<link>
    <visual>
        <geometry></geometry>
        <origin></origin>
        <material></material>
    </visual>
</link>
```

```
<joint name="joint1" type ="revolute">
    <parent link="link1">
    <child link="link2">
    <origin xyz="1 0 0" rpy="0 0 0">
    <axis xyz="0 1 0">
</joint>
```

# MuJoCo XML Format (MJCF)

## URDF does not include information about how motor makes motion

```
  <body name="base" pos="0 0 0.1">
    <joint type="free"/>  <!-- 자유 공간 이동 -->
    <geom type="box" size="0.25 0.25 0.05" rgba="0 0 1 1"/>
    <inertial pos="0 0 0" mass="10" diaginertia="1 1 1"/>

    <!-- 팔 바디 -->
    <body name="arm" pos="0 0 0.05">
      <joint name="arm_joint" type="hinge" axis="0 0 1" range="-1.57 1.57"/>
      <geom type="cylinder" size="0.05 0.15" rgba="1 0.5 0 1"/>
      <inertial pos="0 0 0.15" mass="2" diaginertia="0.1 0.1 0.01"/>
    </body>
  </body>
<actuator>
  <position name="arm_actuator" joint="arm_joint" kp="50"/>
</actuator>
```

# Online tutorials

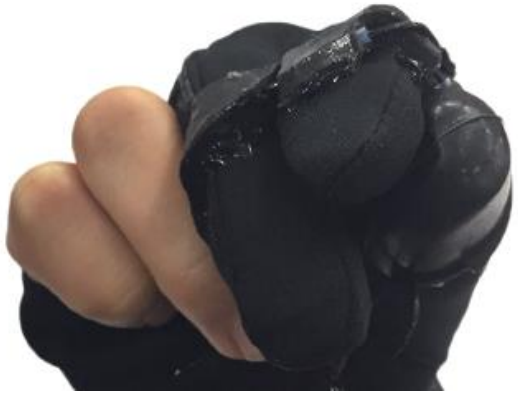
- **Issac Sim**
  - Official: https://docs.isaacsim.omniverse.nvidia.com/5.1.0/index.html
  - Korean Version: https://wikidocs.net/295725
  - Gr00T: https://wikidocs.net/book/17847
- **Genesis**
  - Official: https://genesis-world.readthedocs.io/en/latest/user_guide/getting_started/locomotion.html
  - Manipulation: https://genesis-world.readthedocs.io/en/latest/user_guide/getting_started/manipulation.html
- **Locomotion (Reinforcement Learning)**
  - https://github.com/leggedrobotics/legged_gym

# 3. Case study – how can we use simulation in research

# GRIPIT: Writing Assistive Device for People with Stroke/SCI

## I want to write because I have a short-term memory loss.

Kim et al., 2017



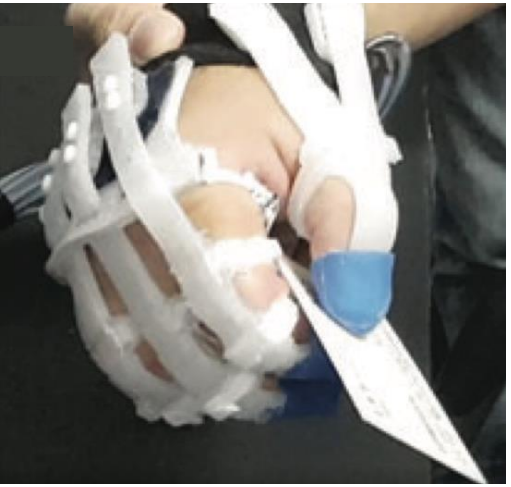
L군(27, 대학생)
교통사고로 인한 뇌출혈
2년여간의 혼수상태
오른손 뇌병변 편마비
단기 기억 상실

왼손을 사용할 수 있지만
오른손 사용을 포기하고싶지 않아

단기 기억 상실을 극복하기 위해
필기를 더 하려고 노력

# Tendon-Driven Hand Wearable Robots

## For assist or rehabilitate the people with spinal cord injury or stroke



Kim et al., 2022

Kim et al., 2025

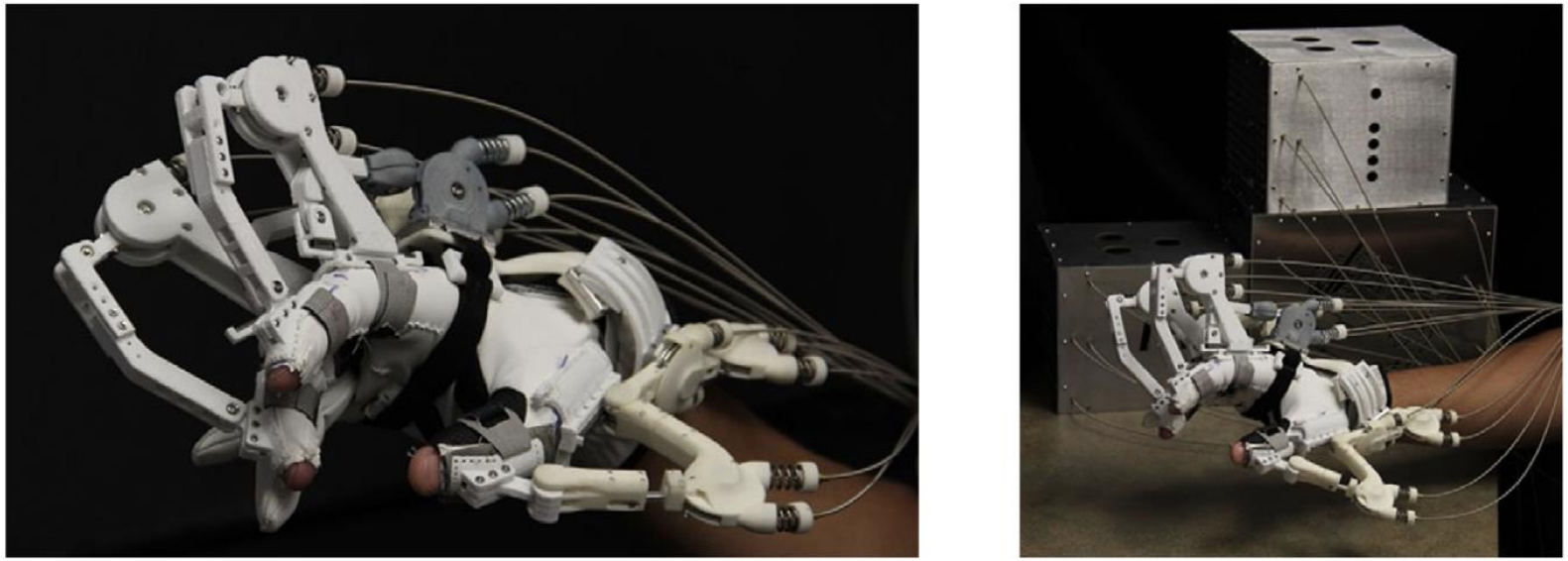
Yun et al., 2017

Lee et al., 2014

- **Actuator can be located away from the wearing part by using tendon-transmission.**
- **Transmitting tension to the human joint torque is important.**

# Challenge 1: Hand motion requires excessive number of motors

## Hand motion requires excessive number of motors



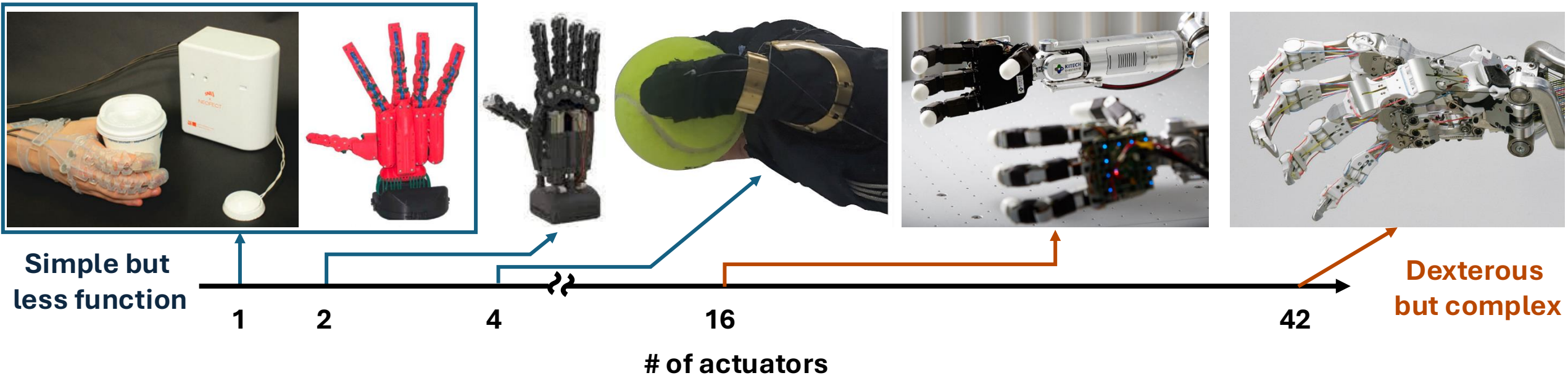
(a) (b)

- Assisting whole finger joints make the robot system bulky
- The size and cost of motor/driver is also not negligible (see boxes in figure b)

Yun, Youngmok, et al. "Improvement of hand functions of spinal cord injury patients with electromyography-driven hand exoskeleton: A feasibility study." *Wearable Technologies* 1 (2020): e8.

# Challenge 1: Hand motion requires excessive number of motors

- Trade-off between the functionality and the simplicity/price



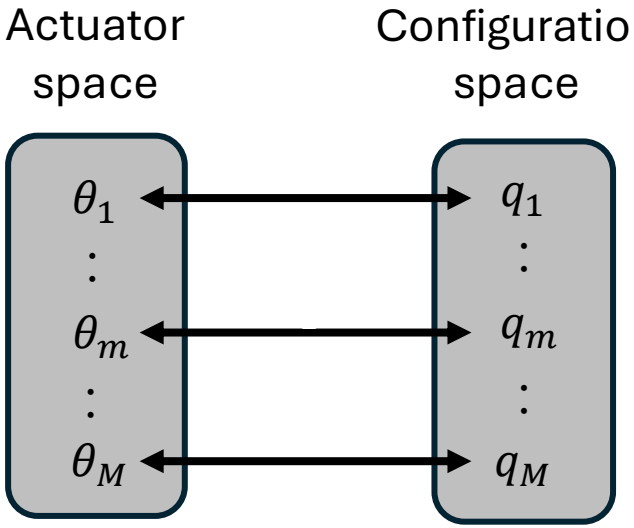

**How can we find the optimal number of actuators?**

**How can we route the tendons appropriately?**

# Difference between tendon-driven robots and the others

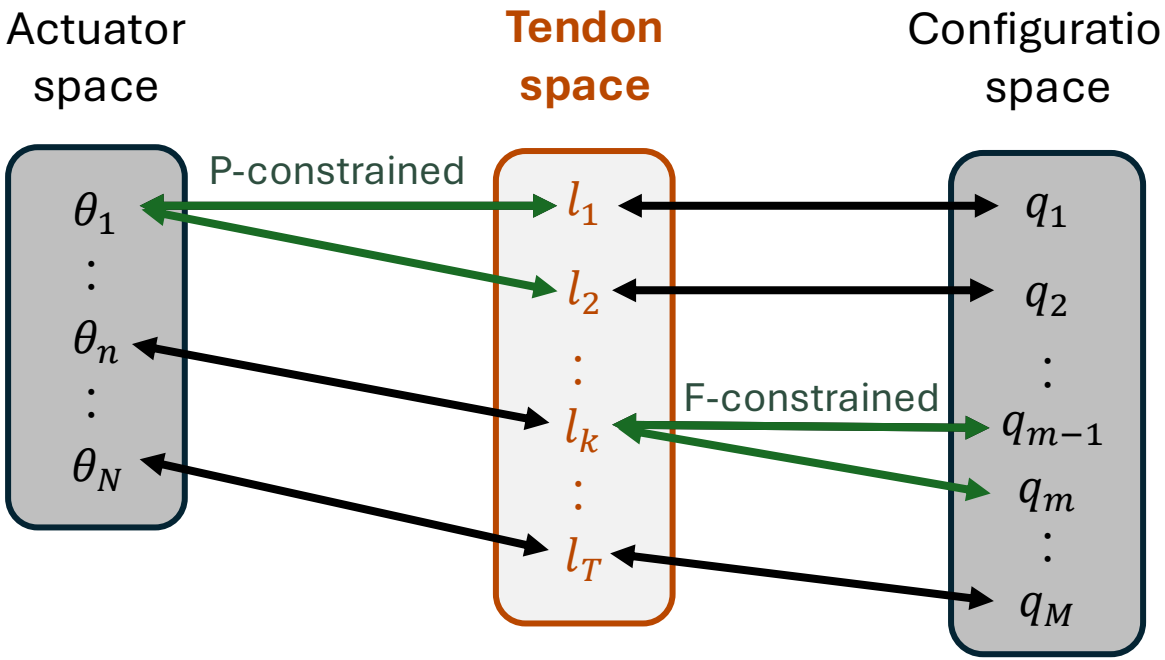
## Additional space between actuator and joint space

### Moving robots using actuators (Full DOF robots)




$$f(\boldsymbol{\theta}) = \boldsymbol{q}$$
$$\boldsymbol{J}\dot{\boldsymbol{\theta}} = \dot{\boldsymbol{q}}$$

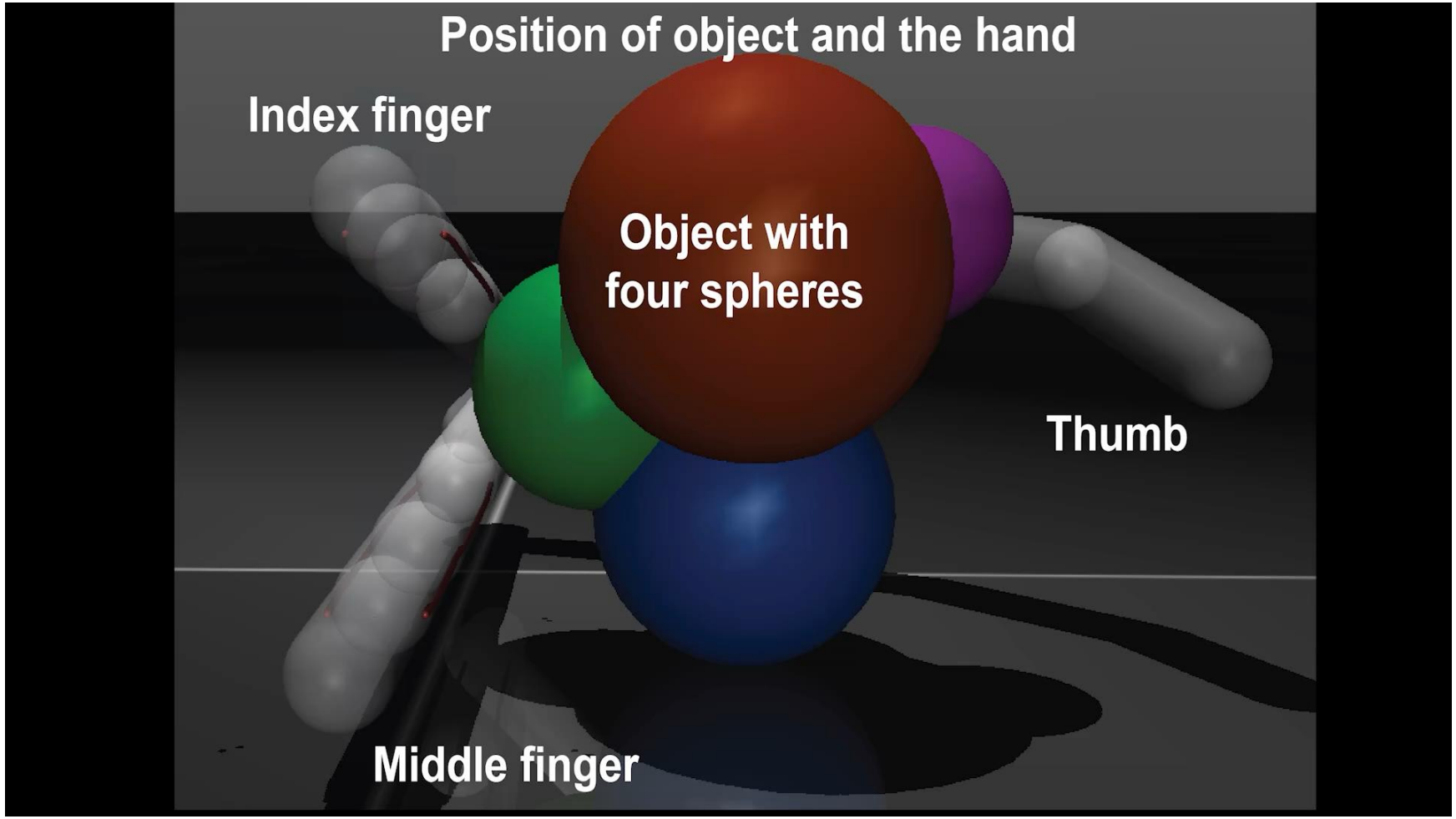
### Under-actuated Tendon-driven robots




**B. Kim**, U. Jeong, and K.-J. Cho,"Exo-Glove Pinch: Soft Hand-wearable Robot Designed through the Analysis of the Constrained Tendon Routing," IEEE Robotics and Automation Letters, submitted.

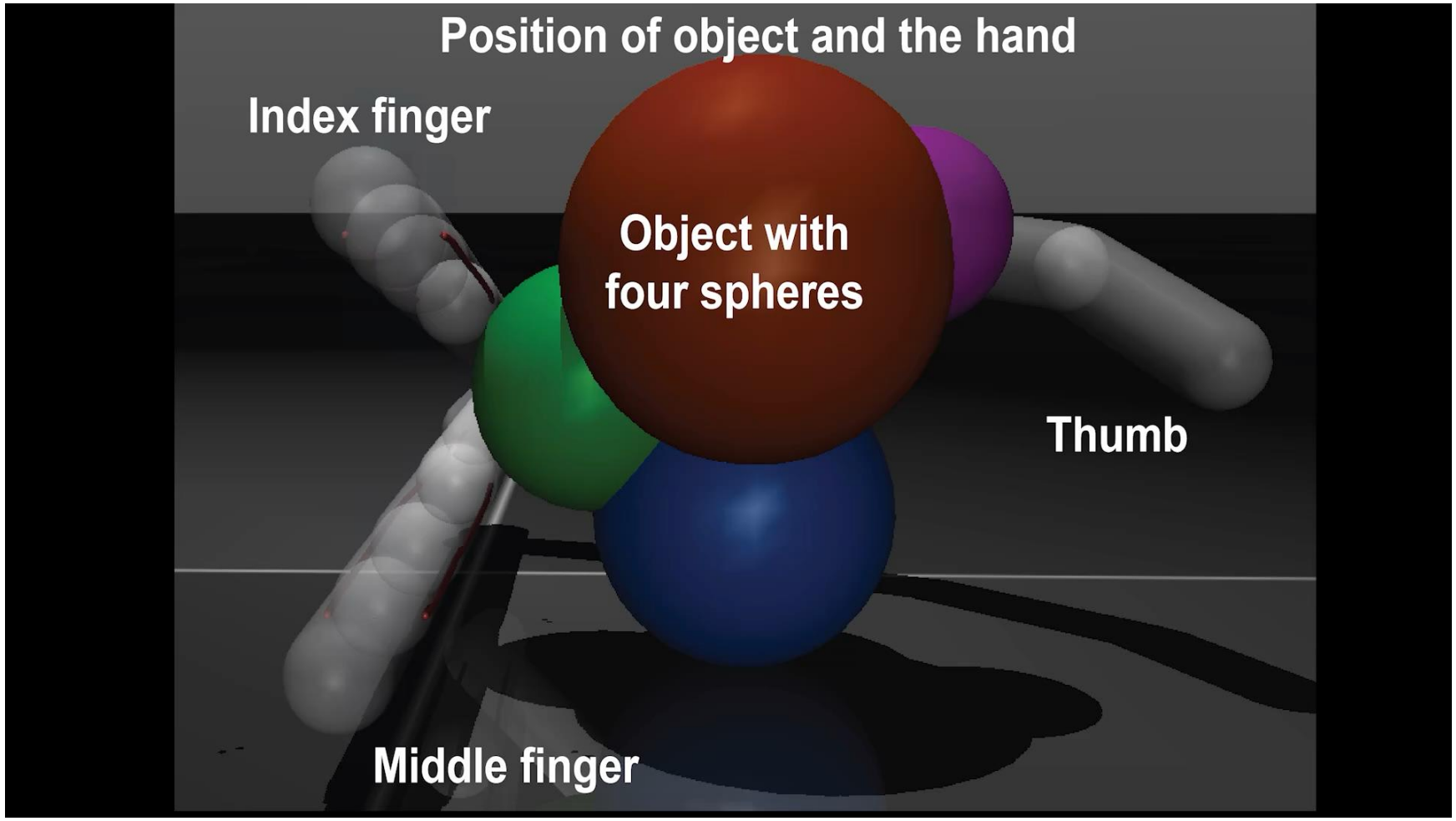
# Using MuJoCo to simulate the grasp performance

## When using the position constraint



# Using MuJoCo to simulate the grasp performance

## When using the Force constraint



# Benefits of using simulation (even if it is not robot learning)

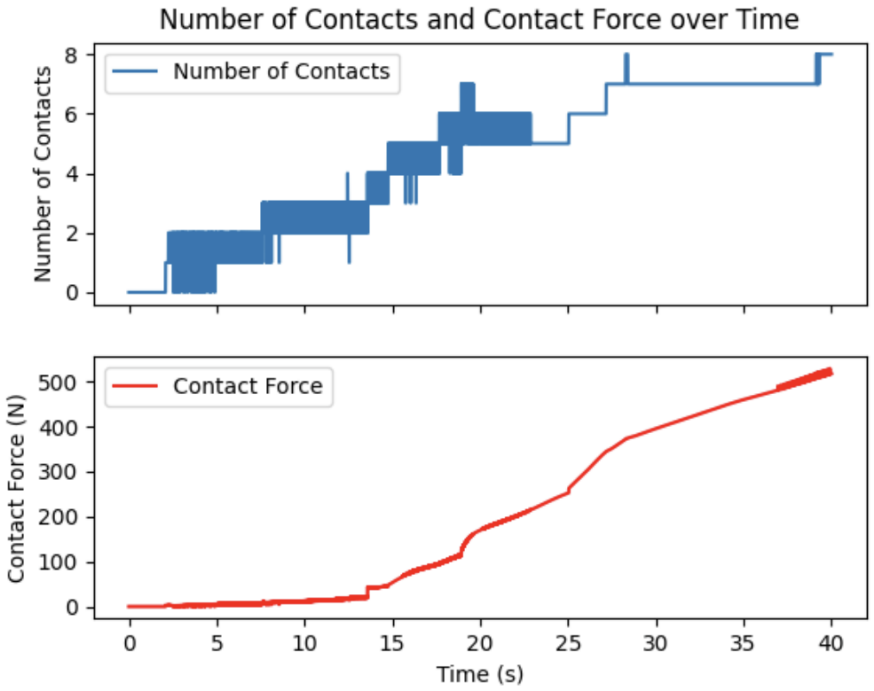



**Fig. S2.1.3.** Number of contact points (CP) and the scalar summation of the contact force (CF) when FTR is used.

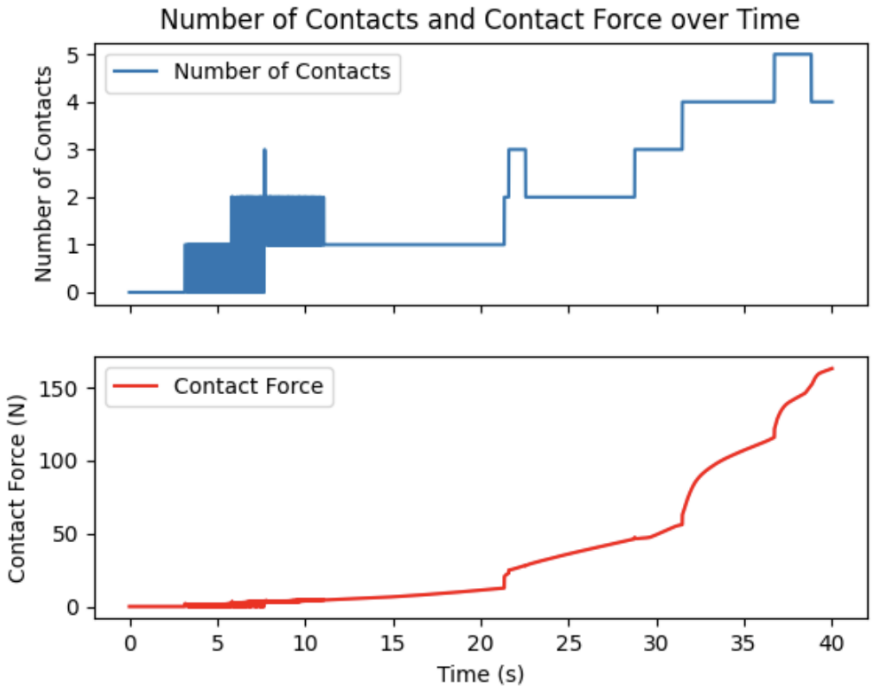



**Fig. S2.1.4.** Number of contact points (CP) and the scalar summation of the contact force (CF) when PTR is used.

- **We can obtain performance which is hard to measure in the real-world.**